MULTI-TENANT AI CHATBOT SAAS · TECHNICAL REFERENCE

Build Your Own AI Chatbot SaaS

Deep-Dive Technical Guide — every file, endpoint, query, and prompt explained

A $0-infrastructure, multi-tenant AI chatbot platform with RAG knowledge bases, appointment booking, and an embeddable widget. This guide walks the actual code: 24 API routes, 7 tables plus a vector index, 4 deploy workflows, 11 secrets, every LLM prompt — with known gaps and one-line fixes called out honestly.

Audience — developers & curious operators **Version** — v1.0.0 **Date** — September 2026

Reading time — ~90 min

WHAT'S INSIDE

Project: `chatbot-saas/` · Free stack: Cloudflare · Google Gemini · Turso · Clerk · Google Apps Script — **$0/month, no credit card required**

Contents

All 13 chapters. Every entry is a link — click to jump, in the PDF or in your browser. Page numbers are the printed page numbers.

⚠ **STACK UPDATE (SEPTEMBER 2026) — HOW THIS GUIDE MAPS TO THE CURRENT CODE.** This guide documents the *original* system, whose backend ran as a **DOCKER SPACE ON HUGGING FACE** (FastAPI + Ollama + nginx under supervisord). Hugging Face moved Docker/Gradio Spaces behind PRO in July 2026, so the production backend is being ported to a **CLOUDFLARE WORKER** (`worker/`, TypeScript) that calls the **GOOGLE GEMINI** API — see `hf-docker-exit-spec.md` and the Appendix "Migration Roadmap" at the end of this guide. Chapters 3–13 still describe the *reference implementation* in `backend/` (kept in the repo while the port proceeds), so their API/prompt/database detail remains accurate for the code you'll read — but any hosting/deploy claim about HF Spaces, Ollama, nginx, supervisord, or keep-alive is **HISTORICAL**. The live deployment steps live in `DEPLOY.md`; where this guide's diagrams still show the old container, read "Cloudflare Worker + Gemini" instead of "HF Space + Ollama".

1. System Architecture

This project is a **multi-tenant AI chatbot platform** that runs on **$0 of infrastructure cost**. A single operator (you) runs one backend, one database, and one widget build, and sells chatbot "seats" to many client businesses. Each client is a *tenant* — their own little slice of the platform with their own name, colors, knowledge base, business hours, and appointments.

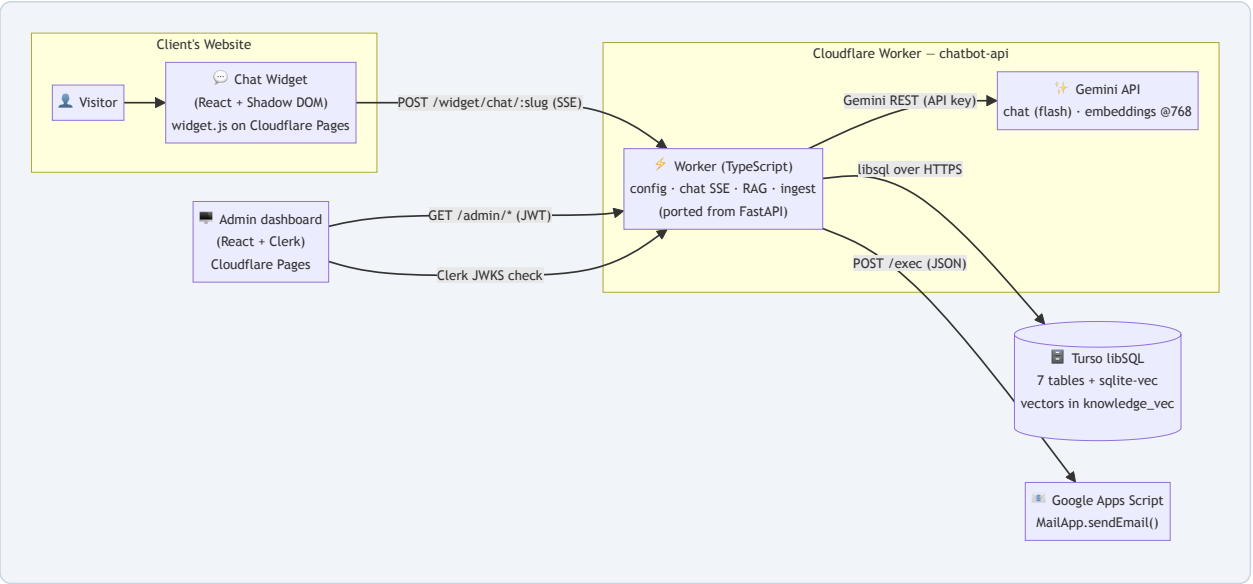
The whole system is built from six free services connected by plain HTTP and SQL. Nothing about it is exotic: one TypeScript API Worker on Cloudflare, one SQLite-compatible remote database, one hosted LLM API (Google Gemini — the original local Ollama runtime was replaced when Docker hosting on Hugging Face became paid), one Gmail-based email shim, and two static React front-ends. The pre-migration Python implementation remains in `backend/` as the reference for the port (see the stack-update note above).

1.1 The six components

#	Component	What it runs	Hosted where	Cost
1	**Widget**	React 18 chat UI compiled to a single `widget.js` (IIFE) mounted inside a Shadow DOM	Cloudflare Pages (static file)	Free
2	**Backend API**	Cloudflare Worker (`worker/` , TypeScript): widget config, chat (SSE), RAG retrieval, knowledge ingest, Clerk JWKS auth — ported 1:1 from the FastAPI app in `backend/app` (24 endpoints)	Cloudflare Workers (`chatbot-api`)	Free (100k req/day)
3	**AI runtime**	Google Gemini API: flash-class chat model (`CHAT_MODEL`) + `gemini-embedding-001` @768-dim embeddings (was Ollama `phi3:mini` / `nomic-embed-text`)	Google (API key; called from the Worker)	Free tier (volatile — see spec §5.5)
4	**Database**	Turso (libSQL) with the `sqlite-vec` extension for vector search; 7 tables + 1 virtual table	Turso cloud	Free (9 GB, 1 B reads/mo)
5	**Email**	Google Apps Script web app (`Code.gs`) that calls `MailApp.sendEmail()`	Google's servers	Free (100 emails/day)
6	**Admin dashboard**	React 18 SPA with Clerk authentication; manages tenants, knowledge, analytics	Cloudflare Pages (static file)	Free

> **CLOUDFLARE WORKERS AND PAGES — UPDATED.** In the current stack Cloudflare plays both roles: **WORKERS** runs the backend API (`worker/` → `chatbot-api`), and **PAGES** serves the static `widget.js` + admin SPA. CORS and the widget `/widget/*` routes are handled in the Worker itself (no nginx). The widget's `wrangler.toml` (Pages output) is separate from the API Worker's config. Rate limiting in the old design lived in nginx inside the HF Space; in the Worker it's per-tenant self-throttling (spec §5.7).

1.2 Architecture diagram



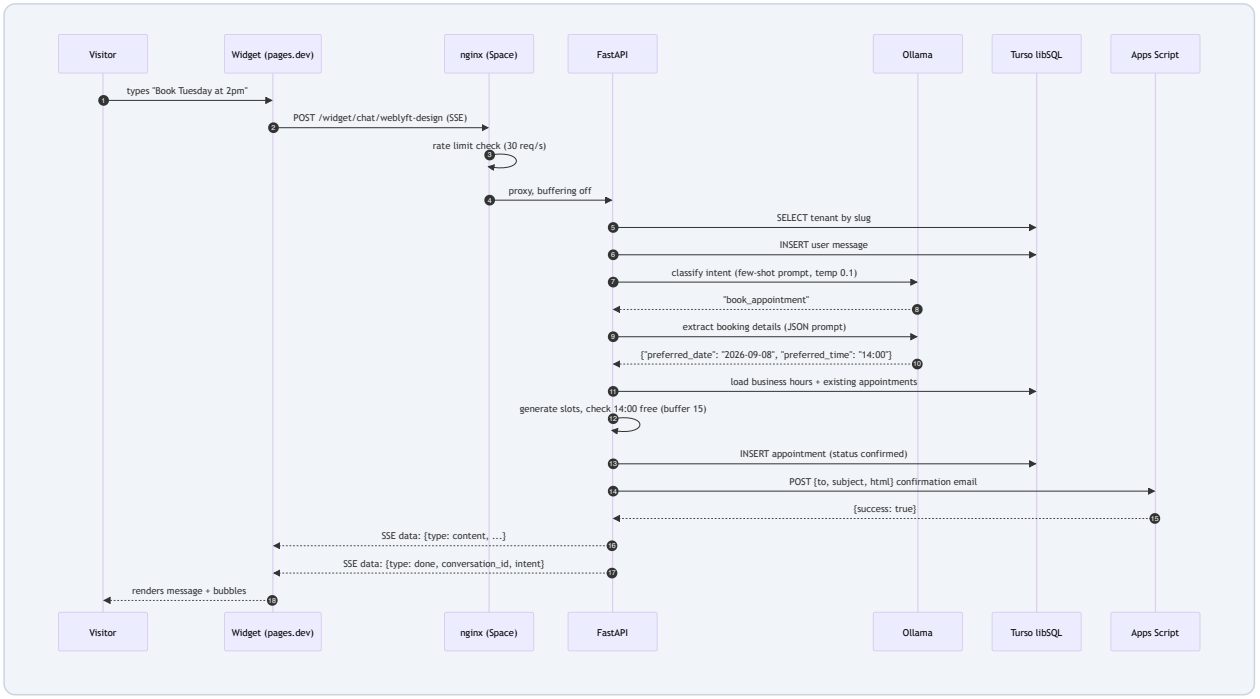
1.3 Component responsibilities

Component	Responsibilities	Key files
Widget	Reads `data-tenant` from its `<script>` tag, fetches per-tenant config (colors, greeting, bot name, business hours), streams chat via SSE, renders clickable appointment slots, persists session + conversation IDs in `localStorage`	`widget/src/main.tsx` , `ChatWidget.tsx` , `hooks/*` , `components/*`
Worker (was nginx + FastAPI)	CORS, routing, per-tenant self-throttling, tenant resolution, intent classification, RAG retrieval, chat SSE, email dispatch via GAS, Clerk JWKS verification for admin routes; the same responsibilities lived in nginx+FastAPI in the old stack	`worker/src/**` (reference: `backend/app/**`)
Gemini (was Ollama)	Flash-class chat model for classification/extraction/synthesis and `gemini-embedding-001` @768-dim embeddings for RAG; free-tier daily caps handled gracefully (spec §5.7)	`worker/src/llm.ts` (reference: `backend/app/services/llm.py`)
Turso	Stores tenants, users, conversations, messages, appointments, knowledge chunks (+ vector index), usage logs; executes vector similarity search via `vec_distance_cosine`	`database.py` , `migrations/001_initial_schema.sql`
Apps Script	Receives `{to, subject, html}` JSON, sends via Gmail, returns `{success: true}` ; run-as-me authorization means emails come from your Gmail address	`gas-email/Code.gs`
Admin dashboard	Clerk sign-in, tenant CRUD, knowledge upload (PDF/TXT/FAQ), conversation and appointment tables, business-hours editor, analytics cards	`admin-dashboard/src/**`

GitHub Actions	Deploys the Worker (`deploy-worker.yml` : typecheck → `wrangler deploy` → secrets) and widget + admin to Cloudflare Pages; no keep-alive needed (Workers don't sleep)	`.github/workflows/*.yml`

1.4 One message, end to end

The single most important path in the system. A visitor types *"Book Tuesday at 2pm"* and this is everything that happens:



Because the chat endpoint streams with **Server-Sent Events** (SSE), the visitor sees the bot's answer being typed out token by token, exactly like ChatGPT. The `done` event carries the `conversation_id` so the widget can continue the same thread (Chapter 8).

1.5 Tech stack rationale — and what was rejected

Layer	Chosen	Why	Alternatives considered	Why not
API framework	FastAPI (Python)	Async `await db.execute()` fits remote-latency libSQL calls; Pydantic gives free request validation; SSE via `StreamingResponse` is one function; huge ML ecosystem (fitz, ollama) in the same language as the AI stack	Express (Node), Django, Next.js API routes	Node would force a second language for AI/PDF work; Django is sync-first and heavier; Next.js pulls in a full React app where we only need an API

Database	Turso (libSQL = SQLite fork)	SQLite semantics (JSON-in-TEXT, unix timestamps) keep the schema trivial; the `sqlite-vec` extension gives vector search without a separate vector database; generous free tier (9 GB, 1 B reads/mo); zero-ops	Postgres + pgvector (Supabase/Neon), PlanetScale, MongoDB	pgvector needs a managed Postgres (heavier free tier, more moving parts); a document DB loses joins for conversations/messages/appointments
LLM	Ollama, `phi3:mini` + `nomic-embed-text`	Runs *inside* the free HF Space — no per-token API cost, no API key, private data stays local; 3.8B model fits in 16 GB RAM and answers fast enough on CPU	OpenAI API, Anthropic, Gemini	Per-token cost breaks the "$0 forever" premise; data would leave the platform; needs a credit card
Hosting	Cloudflare Worker (`worker/`) + Cloudflare Pages	Worker = the ported backend as TypeScript (never sleeps, 100k req/day free); Pages = free static hosting with unlimited bandwidth for the two front-ends; all three auto-deploy from GitHub Actions. (The original Docker-on-HF hosting is gone — HF made Docker Spaces paid in July 2026.)	Vercel, Netlify, Railway, Fly.io	Vercel/Netlify have no always-free compute backend; Railway/Fly have free tiers but with sleep/wake and credit-card walls; Workers + Pages keeps *everything* free in one account — with Gemini for the LLM instead of a self-hosted container
Auth	Clerk	Hosted sign-in UI (email + password), JWT sessions, React SDK (`<SignedIn>`) and a server-side verify path via `python-jose` against Clerk's JWKS public keys	Auth0, Firebase Auth, roll-your-own JWT	Auth0's free tier is stingier; Firebase pulls in Google Cloud; hand-rolled auth is the #1 security mistake — Clerk's 10k MAU free tier covers a solo operator forever
Email	Google Apps Script + Gmail	Zero setup cost, zero API keys; the "web app" is a real HTTPS endpoint that FastAPI POSTs JSON to; Gmail's 100/day free quota is plenty for a booking business	SendGrid, Resend, Postmark, SES	All need a credit card and a verified sending domain; transactional providers charge per email at this scale; Gmail is already universal
Widget distribution	Single IIFE file on Cloudflare Pages	Clients add one `<script>` tag; the widget is self-contained (React bundled in), styles isolated by Shadow DOM, so it can't clash with the client's CSS	npm package, iframe embed, Web Component	npm install is unrealistic for non-technical clients; iframes are ugly and block modals; a raw Web Component is basically what Shadow DOM gives us anyway

> **WHY THIS ARCHITECTURE WINS AT $0.** The key move is *concentration*: one free Docker container holds the API, the web server, and the AI. The only remote dependency is Turso, which speaks SQL over HTTPS — no VPC, no private networking, no NAT. Every component can be torn down and rebuilt from GitHub in minutes.

1.6 Multi-tenancy model

Tenancy is the *raison d'être* of the schema. Each client business is a row in `tenants`, identified by a human-readable `slug` (e.g. `weblyft-design`). Three layers key off the slug:

- **Widget layer:** the embed tag carries `data-tenant="weblyft-design"`; every API call includes the slug in the URL path (`/widget/chat/weblyft-design`).
- **Middleware layer:** `TenantMiddleware` extracts the slug from the path, the `X-Tenant-Slug` header, or a subdomain, then loads the tenant row into `request.state` (Chapter 4.2).
- **Data layer:** every data table (`end_users`, `conversations`, `appointments`, `knowledge_chunks`, `usage_logs`) carries a `tenant_id` foreign key — `messages` is the one exception, reaching its tenant only through the owning `conversations` row — and every query filters on it, a hard isolation boundary that prevents client A's chat history from leaking into client B's dashboard (Chapter 3).

Isolation is enforced in *every* query, not just at the edge. For example, conversation reads verify `WHERE id = ? AND tenant_id = ?`; vector search embeds `WHERE kc.tenant_id = ?` into the similarity query itself. This is the pattern to preserve if the platform grows.

2. Infrastructure & Deployment

Updated: everything now ships as a **Cloudflare Worker** (`worker/` , deployed by `deploy-worker.yml`) plus two static builds on Cloudflare Pages. The content below documents the *original* Docker deployment (Dockerfile, nginx, supervisord, HF registry workflows) — kept as history since those files were removed and the old pipeline no longer exists. For the current deploy pipeline, read `DEPLOY.md` and `worker/wrangler.toml` ; this chapter is the "how it used to be" reference.

2.1 Dockerfile — line by line

```
FROM python:3.11-slim                       # 1. ~120 MB base image, Debian bookworm, no build tools

RUN apt-get update && apt-get install -y --no-install-recommends \\
    curl \\                                  # 2. needed for the Ollama installer and health checks
    ca-certificates \\                       # 3. TLS trust so pip / ollama / libsql can reach the internet
    supervisor \\                            # 4. process manager (runs ollama + fastapi + nginx)
    nginx \\                                 # 5. reverse proxy / rate limiter / SSE enabler
    && rm -rf /var/lib/apt/lists/*          # 6. shrink image: drop apt metadata

RUN curl -fsSL https://ollama.com/install.sh | sh   # 7. installs ollama CLI + server binary

WORKDIR /app                                 # 8. everything below is relative to /app

COPY backend/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt   # 9. Python deps (fastapi, libsql-client, ollama,
fitz...)

COPY backend/app ./app                       # 10. the FastAPI package
COPY backend/alembic.ini .
COPY backend/migrations ./migrations         # 11. SQL migration files

COPY nginx.conf /etc/nginx/nginx.conf        # 12. replace default nginx config entirely
COPY supervisord.conf /etc/supervisor/conf.d/supervisord.conf  # 13. our 3-program supervisor config

RUN mkdir -p /var/log/supervisor /var/run/ollama /root/.ollama   # 14. dirs supervisord/ollama expect

EXPOSE 80 443 8000 11434                     # 15. documents ports; actual mapping done by the platform

HEALTHCHECK --interval=30s --timeout=10s --start-period=60s --retries=3 \\
    CMD curl -f http://localhost:8000/health || exit 1   # 16. Docker-level liveness probe

CMD ["/usr/bin/supervisord", "-c", "/etc/supervisor/conf.d/supervisord.conf"]  # 17. PID 1
```

Line	Why it matters
1	`python:3.11-slim` keeps the image small; 3.11 is old enough to be battle-tested with all pinned deps (see `requirements.txt` pins like `fastapi==0.109.0`).
2–6	Installing `nginx` and `supervisor` from apt is the fastest way to get both into the image. `--no-install-recommends` + cleanup keeps the final image lean.
7	The official Ollama installer puts `ollama` at `/usr/local/bin/ollama` (supervisord references this exact path).
9	`pip install` happens at image-build time so the container *starts* fast — only the **models** (phi3:mini ≈ 2.2 GB, nomic-embed-text ≈ 274 MB) download at first boot, not the Python packages.

10–11	The image contains `app/`, `migrations/`, `alembic.ini`, and `run_migrations.py`. supervisord runs the migration script before uvicorn (2.2), so a fresh database is migrated automatically on first boot.
16	Docker-level liveness probe. FastAPI serves `GET /health` directly (main.py), so this works as configured.
17	supervisord as PID 1 is what keeps three processes alive and restarts them on crash — a bare `CMD uvicorn` would die the moment Ollama or nginx exits.

> **HEALTH ENDPOINTS.** FastAPI serves both `GET /health` (an alias in `main.py`) and `GET /widget/health` (widget router). The Docker `HEALTHCHECK`, the nginx `/health` block (2.3), and the keep-alive workflow (2.6) all work as written.

2.2 supervisord.conf — the process trio

```
[supervisord]
nodaemon=true                          # run in foreground (Docker requires PID 1 to stay alive)
logfile=/var/log/supervisor/supervisord.log
pidfile=/var/run/supervisord.pid
childlogdir=/var/log/supervisor

[program:ollama]
command=/usr/local/bin/ollama serve     # starts the HTTP server on 0.0.0.0:11434
environment=OLLAMA_HOST=0.0.0.0:11434,OLLAMA_MODELS=/root/.ollama
autorestart=true
stdout_logfile=/var/log/supervisor/ollama.log        # per-program logs
stderr_logfile=/var/log/supervisor/ollama_err.log    # (Chapter 11.5 shows which to tail)
priority=10                            # starts FIRST (models must be ready before API calls)

[program:fastapi]
command=uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 1
directory=/app
autorestart=true
stdout_logfile=/var/log/supervisor/fastapi.log
stderr_logfile=/var/log/supervisor/fastapi_err.log
priority=20                            # second

[program:nginx]
command=nginx -g "daemon off;"         # "daemon off" keeps nginx in the foreground for supervisord
autorestart=true
stdout_logfile=/var/log/supervisor/nginx.log
stderr_logfile=/var/log/supervisor/nginx_err.log
priority=30                            # last (proxies to fastapi, so it must exist first)
```

> **WHY `--workers 1`?** Free Spaces get 2 vCPUs, but the AI calls are synchronous and CPU-bound inside Ollama — more uvicorn workers would contend for the same cores, and each worker would hold its own async libSQL connection. One worker also keeps SSE event ordering trivially correct. (See Chapter 13 for the multi-worker upgrade path.)

2.3 nginx.conf — the front door

nginx is the only process exposed to the internet. It does five jobs: **rate limiting**, **security headers**, **proxying**, **SSE friendliness**, and **gzip**.

Rate limiting zones

```
# $binary_remote_addr = the visitor's IP, in compact binary form (4 bytes for IPv4)
limit_req_zone $binary_remote_addr zone=widget_limit:10m rate=30r/s;   # chat & widget traffic
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=60r/s;      # admin & tenant API
```

Each zone is a shared 10 MB in-memory table of IP→counter. `30r/s` means a sustained 30 requests per second; bursts up to the `burst` parameter are queued.

Location	Rate limit	Special config
`/health`	none	no access log — but FastAPI serves health at `/widget/health` (see the warning in 2.1); this block currently forwards plain `/health` to a 404
`~ ^/widget/config/`	`widget_limit burst=10 nodelay`	standard proxy headers
`~ ^/widget/chat/`	`widget_limit burst=5 nodelay`	**SSE block:** `proxy_http_version 1.1` , `Connection ""` , `proxy_buffering off` , `proxy_cache off` , read/send timeouts 300 s
`~ ^/widget/`	`widget_limit burst=10 nodelay`	availability + appointment booking
`~ ^/admin/`	`api_limit burst=20 nodelay`	—
`~ ^/api/`	`api_limit burst=20 nodelay`	tenant dashboard routes
`/ollama/`	none	internal debugging passthrough to `127.0.0.1:11434`
`/`	none	catch-all → FastAPI root

> **WHY `PROXY_BUFFERING OFF` ON THE CHAT ROUTE IS NON-NEGOTIABLE.** SSE is a long-lived stream: FastAPI sends `data: {...}\n\n` chunks over minutes. If nginx buffers, tokens pile up server-side and the visitor sees nothing until the stream ends — destroying the streaming UX. Disabling buffering + `Connection ""` (HTTP/1.1 keep-alive upstream) + a 300 s timeout keeps each token flowing through immediately.

Security headers applied globally (`add_header ... always`): `X-Frame-Options: SAMEORIGIN` (blocks clickjacking of the API), `X-Content-Type-Options: nosniff` , `X-XSS-Protection` , and `Referrer-Policy: strict-origin-when-cross-origin` . Upstream keep-alive pools (`keepalive 32` / `16`) reuse TCP connections to FastAPI and Ollama, which matters for chat latency.

2.4 Hugging Face Spaces internals

- **Docker SDK Spaces** build your image from the repo's `Dockerfile` on every push (or on demand). The Space's public URL — `https://USERNAME-SPACENAME.hf.space` — terminates TLS and routes to the container's exposed ports; external traffic arrives on port 80 (nginx).
- **Env vars:** set once in Space settings → Repository secrets; the same 11 names used by GitHub Actions (the app reads them via `pydantic-settings` , Chapter 4.1).

- **Free hardware:** 16 GB RAM / 2 vCPU, CPU-only. Phi-3-mini comfortably fits with nomic-embed-text alongside FastAPI + nginx.
- **First boot:** the models are *not* in the image — a small entry script (or a manual `ollama pull` in the Space terminal) downloads them once. Budget 5–10 minutes of boot time on first deploy; subsequent cold starts are faster because Ollama caches models in `/root/.ollama`.
- **Sleep behavior (free tier):** after ~48 h of no traffic the Space suspends; the first request then takes ~30 s to wake. The keep-alive workflow (2.7) prevents this in practice.
- **Terminal:** Settings → Open in terminal gives a shell *inside the running container* — good for `ollama pull`, log inspection, and (once the image note in 2.1 is applied) the CLI scripts and migrations.

2.5 Cloudflare Pages config

```
# widget/wrangler.toml (and admin-dashboard/wrangler.toml, same shape)
name = "chatbot-widget"
compatibility_date = "2024-01-01"
pages_build_output_dir = "dist"     # Cloudflare serves the contents of dist/

[build]
command = "npm run build"           # run by Pages on every deploy
```

Both front-ends are pure static builds served from a `dist/` folder: `widget/dist/` (one IIFE bundle — see 8.6 for the real output file names and the one-time build fixes) and `admin-dashboard/dist/` (index.html + hashed assets). No server-side logic runs at the edge. Deployment happens two ways: the GitHub Action `cloudflare/pages-action@v1` (2.6), or a direct Pages git integration (an alternative documented in DEPLOY.md — if you use both, pick one to avoid double deploys).

> **ONE INTEGRATION DETAIL WORTH KNOWING.** The widget calls *relative* URLs (`/widget/chat/{slug}`), so in production the built widget must be served from the same origin as the API — either by proxying `/widget/*` on the widget's pages.dev host to `*.hf.space` (a Pages Function or a proxy rule), or by serving the file from the Space itself. Chapter 12 covers the symptom (*"widget says failed to fetch"*) and the fix.

2.6 GitHub Actions — the four workflows

deploy-hf.yml — backend to HF Spaces (Docker push)

```
on:
  push:
    branches: [main]
    paths: ['backend/**', 'Dockerfile', 'supervisord.conf', 'nginx.conf', '.github/workflows/deploy-
hf.yml']
  workflow_dispatch:                      # manual "Run workflow" button

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4         # get the repo
      - uses: docker/setup-buildx-action@v3   # enable BuildKit (multi-stage caching)
      - uses: docker/login-action@v3
        with:
          registry: registry.hf.space
          username: ${{ secrets.HF_USERNAME }}
          password: ${{ secrets.HF_TOKEN }}   # HF write token acts as the registry password
      - uses: docker/build-push-action@v5
        with:
          context: .
          push: true
          tags: registry.hf.space/${{ secrets.HF_USERNAME }}/${{ secrets.HF_SPACE_NAME }}:latest
          cache-from: type=registry,ref=.../buildcache     # pull layer cache
          cache-to: type=registry,ref=.../buildcache,mode=max  # push cache → fast rebuilds
      - run: |
          curl -X POST "https://huggingface.co/api/spaces/.../restart" \\
            -H "Authorization: Bearer ${{ secrets.HF_TOKEN }}"   # force the Space to pick up the new image
```

Key points: the `paths` filter means a widget-only commit doesn't rebuild the 2 GB image; registry caching turns repeat deploys into minutes instead of tens of minutes; the final `restart` API call is what actually swaps the running Space to the new image.

deploy-widget.yml / deploy-admin.yml — static sites

```
on:
  push:
    branches: [main]
    paths: ['widget/**', '.github/workflows/deploy-widget.yml']   # or 'admin-dashboard/**'
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-node@v4
    with: { node-version: '20' }
  - run: cd widget && npm install       # no lockfile in the repo, so npm ci would fail
  - run: cd widget && npm run build    # → widget/dist/widget.js (see 8.6)
  - uses: cloudflare/pages-action@v1
    with:
      apiToken: ${{ secrets.CLOUDFLARE_API_TOKEN }}
      accountId: ${{ secrets.CLOUDFLARE_ACCOUNT_ID }}
      projectName: chatbot-widget      # must match a Pages project you created
      directory: widget/dist
      branch: main
```

keep-alive.yml — free-tier life support

```
on:
  schedule:
    - cron: '*/30 * * * *'   # every 30 minutes, UTC
jobs:
  keep-alive:
    steps:
      - run: |
          curl -f -s -o /dev/null -w "%{http_code}" \\
            https://${{ secrets.HF_USERNAME }}-${{ secrets.HF_SPACE_NAME }}.hf.space/health \\
            || echo "Space might be sleeping, that's OK"
```

Free Spaces sleep after ~48 h idle. A ping every 30 min resets that clock, so the bot is effectively always awake. The ping URL (`/health`) is served by FastAPI (2.1), so each ping both wakes the Space and gets a real 200. One caveat: `-f` fails on HTTP errors but the `|| echo` swallows it, so the workflow never shows red for a cold start.

2.7 The 11 GitHub secrets

Secret	Used by	From
`TURSO_DATABASE_URL`	backend (`libsql://...`)	Turso → database → connection details
`TURSO_AUTH_TOKEN`	backend + scripts	Turso
`CLERK_PUBLISHABLE_KEY`	admin SPA + backend JWT audience	Clerk → API keys
`CLERK_SECRET_KEY`	backend `verify_admin`	Clerk
`GAS_WEBAPP_URL`	backend `email.py`	Apps Script deploy URL (.../exec)
`HF_TOKEN`	deploy-hf.yml (registry login + restart)	HF → settings → access tokens
`HF_USERNAME`	deploy-hf.yml + keep-alive URL	HF profile
`HF_SPACE_NAME`	deploy-hf.yml + keep-alive URL	the Space you created
`CLOUDFLARE_API_TOKEN`	deploy-widget/admin	Cloudflare → API tokens (Pages:Edit)
`CLOUDFLARE_ACCOUNT_ID`	deploy-widget/admin	Cloudflare dashboard sidebar
`ADMIN_EMAIL`	backend admin authorization	your email (must be in Clerk allowlist)

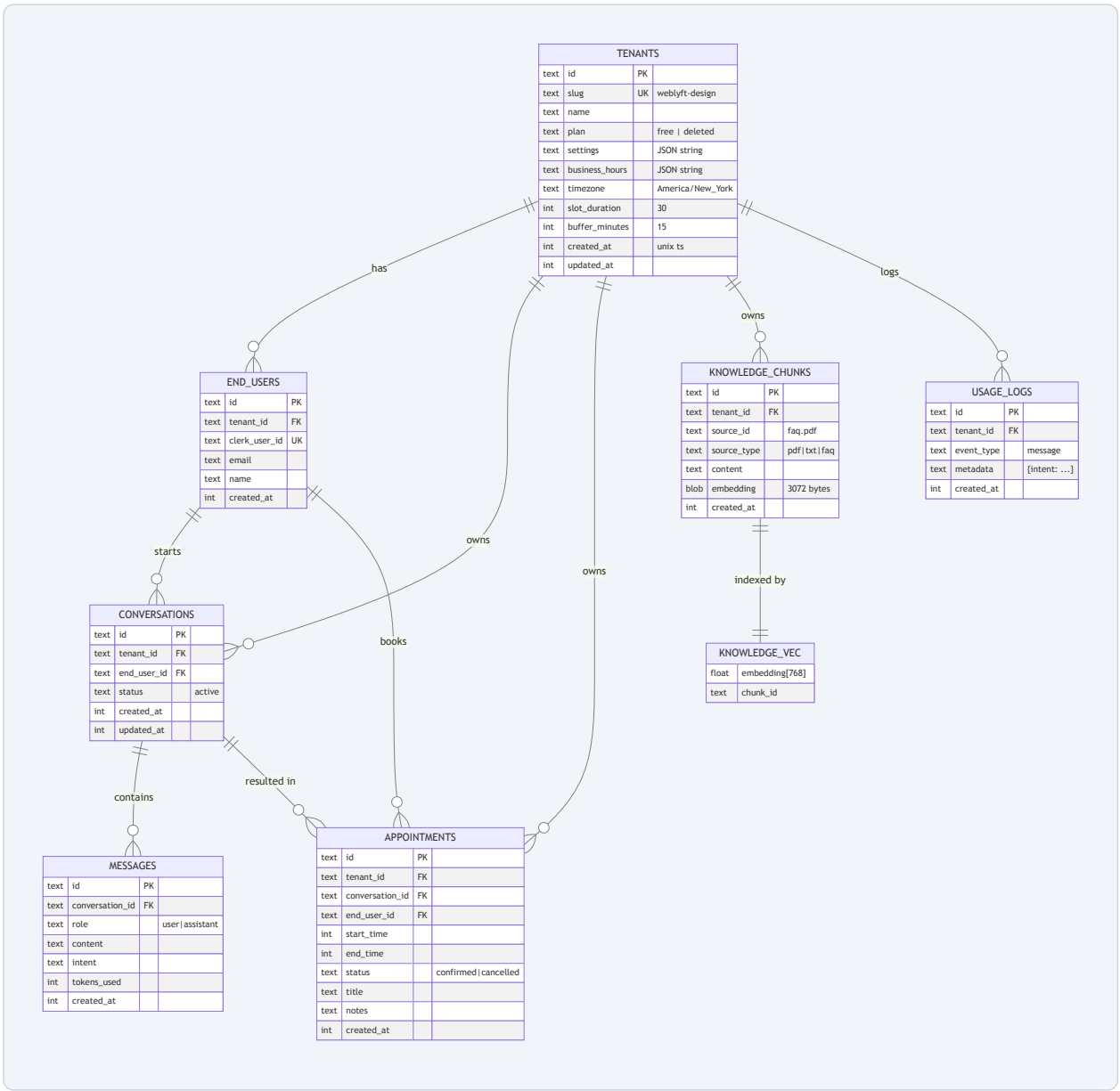
2.8 docker-compose.yml — local development

The compose file runs a standalone `ollama/ollama` container plus the backend image for local testing. The backend talks to **Turso** over HTTPS — there is no local Postgres (an earlier compose draft spun up `pgvector/pgvector:pg16` , which was removed because the app is Turso/libSQL-only; `database.py` imports `libsql_client` and there is no SQLAlchemy Postgres engine anywhere). Set `TURSO_DATABASE_URL` / `TURSO_AUTH_TOKEN` to a real Turso database before `docker compose up` ; the backend command runs `python run_migrations.py` first, then starts uvicorn with live reload.

3. Database Design

Storage is a **Turso database** — a hosted, replicated fork of SQLite called *libSQL* — reached over HTTPS by `libsql-client`. It is augmented by the `sqlite-vec` extension, which adds a real vector index so we can do semantic (meaning-based) search over knowledge-base chunks.

3.1 Entity-relationship diagram



3.2 Table-by-table reference

All primary keys are client-generated UUID strings (no autoincrement), all timestamps are Unix seconds as integers (no timezone ambiguity — UTC epoch), and all flexible/JSON data rides in TEXT columns. That trio — *UUID text PKs, integer*

epoch timestamps, JSON-in-TEXT — is the whole schema philosophy.

Table	Columns	Notes
`tenants`	`id` (PK), `slug` (UNIQUE), `name`, `domain`, `plan` (default `'free'`), `settings` (JSON: bot_name, greeting, colors, logo, show_branding, notification_email), `business_hours` (JSON: per-day open/close + timezone), `timezone`, `slot_duration` (30), `buffer_minutes` (15), `created_at`, `updated_at`	The center of the universe. `plan='deleted'` is a soft delete — rows stay for audit but every query filters `plan != 'deleted'`.
`end_users`	`id`, `tenant_id` FK, `clerk_user_id` (UNIQUE, nullable), `email`, `name`, `metadata`, `created_at`	Website visitors / customers. In the current widget flow, bookings create users with email+name; chat-only sessions fall back to the session id string in `end_user_id`.
`conversations`	`id`, `tenant_id` FK, `end_user_id` FK, `status` ('active'), `metadata`, `created_at`, `updated_at`	One row per chat thread. `updated_at` is bumped after every exchange — that's the sort key for the dashboard's conversation list.
`messages`	`id`, `conversation_id` FK, `role` ('user'|'assistant'|'system'|'tool'), `content`, `intent`, `tool_calls`, `tokens_used`, `created_at`	Immutable chat log. `intent` stores the classification result on assistant messages — cheap analytics later.
`appointments`	`id`, `tenant_id` FK, `conversation_id` FK, `end_user_id` FK, `start_time` (int), `end_time` (int), `status` ('pending'|'confirmed'|'cancelled'|'completed'), `title`, `notes`, `metadata`, `created_at`, `updated_at`	Bookings are pure Unix timestamps — all timezone math happens in Python with `pytz` (Chapter 6), never in SQL.
`knowledge_chunks`	`id`, `tenant_id` FK, `source_id` (file name, e.g. `faq_pdf`), `source_type` ('pdf'|'txt'|'md'|'url'|'faq'), `content`, `embedding` (BLOB), `metadata`, `created_at`	Each chunk = one ~500-word slice of the client's knowledge. The embedding column is the float vector serialized to 3072 raw bytes.
`knowledge_vec`	virtual table: `embedding FLOAT[768]`, `chunk_id TEXT`	The `sqlite-vec` vector index. Both tables are written together and deleted together (`delete_chunks`).
`usage_logs`	`id`, `tenant_id` FK, `event_type`, `metadata`, `created_at`	Append-only event stream; powers the "active tenants" metric and future billing.

3.3 Indexes

Index	Table	Speeds up

`idx_conv_tenant`	conversations(tenant_id)	dashboard conversation list per tenant
`idx_conv_user`	conversations(end_user_id)	"this visitor's threads" lookups
`idx_msg_conv`	messages(conversation_id)	thread history reads
`idx_appt_tenant_time`	appointments(tenant_id, start_time)	availability queries — the hot path (composite index on tenant + start time)
`idx_appt_user`	appointments(end_user_id)	"my appointments"
`idx_knowledge_tenant`	knowledge_chunks(tenant_id)	RAG tenant filter + chunk delete
`idx_usage_tenant_date`	usage_logs(tenant_id, created_at)	usage analytics window queries
`idx_tenant_slug`	tenants(slug)	every widget request resolves slug→tenant; the `UNIQUE` constraint already creates an index — this is a defensive duplicate
`idx_enduser_tenant`	end_users(tenant_id)	"users of tenant X"

3.4 sqlite-vec internals

`sqlite-vec` (v0.1.6 in requirements) is a loadable SQLite extension that adds virtual tables and scalar functions for vector math. Two moving parts matter here:

1. **Loading the extension** — `database.py` runs, on every connect:

   ```
   PRAGMA enable_load_extension = true;
   SELECT vec_version();          -- if the extension is already linked in, this succeeds
   -- fallback if not:
   SELECT load_extension('vec0');
   ```

   The graceful try/except means the app still boots (and degrades to non-vector behavior) if the extension is unavailable in some environment.

2. **The virtual table** — declared in the migration as:

   ```
   CREATE VIRTUAL TABLE knowledge_vec USING vec0(embedding FLOAT[768], chunk_id TEXT);
   ```

   Every chunk's embedding (768 floats for nomic-embed-text) is a row here. The dimension is fixed at table-creation time — changing embedding models later requires rebuilding this table (Chapter 13).

How vectors are serialized

`RAGService` packs 768 Python floats into 3072 raw bytes with `struct` — the exact byte layout libSQL needs for the vector column:

```
def _serialize_vector(self, vector):
    return struct.pack(f'{len(vector)}f', *vector)   # 768 'f' (4-byte floats) → 3072 bytes

def _deserialize_vector(self, data):
    count = len(data) // 4
    return list(struct.unpack(f'{count}f', data))
```

The similarity query

```
SELECT kc.id, kc.content, kc.metadata,
       vec_distance_cosine(kc.embedding, ?) AS distance
FROM knowledge_chunks kc
JOIN knowledge_vec kv ON kc.id = kv.chunk_id
WHERE kc.tenant_id = ?
ORDER BY distance ASC
LIMIT ?;
```

`vec_distance_cosine(a, b)` returns **cosine distance** — a number in [0, 2] where 0 means identical direction. The app converts to similarity with `similarity = 1 - distance`, keeps only chunks above the 0.7 threshold, and caps results at `top_k = 5` (Chapter 5.4 walks the full pipeline).

3.5 Migration SQL walkthrough

The whole schema lives in one idempotent file, `backend/migrations/versions/001_initial_schema.sql`. Every statement is wrapped in `CREATE TABLE IF NOT EXISTS` / `CREATE INDEX IF NOT EXISTS`, so re-running it is safe — which is exactly what `run_migrations.py` does: it splits the file on `;` and executes each statement through the sync libSQL client, printing a truncated copy of each one so you can watch progress. (The SQL *is* copied into the image as `/app/migrations`; only the `backend/run_migrations.py` runner isn't — see 2.1 for where to run it.)

> **WHY RAW SQL INSTEAD OF ALEMBIC AUTOGENERATE?** The models (`SQLModel`) exist mostly for documentation; the real DDL is hand-written because the vector virtual table can't be represented by an ORM, and libSQL's `CREATE VIRTUAL TABLE ... USING vec0(...)` is not SQLAlchemy-friendly. Alembic is configured (`alembic.ini`, `migrations/env.py`) but `env.py` explicitly notes "actual migrations run via scripts" — the pragmatic route.

3.6 Turso / libSQL specifics

- **Two clients, one database:** `database.py` exposes an *async* client (`libsql_client.create_client`) for FastAPI request paths and a *sync* client (`create_client_sync`) for scripts and migrations. Both connect over HTTPS using `TURSO_DATABASE_URL` + `TURSO_AUTH_TOKEN`.
- **Query style:** raw SQL with positional `?` parameters (`db.execute(sql, [params])`), returning a `ResultSet` with `.columns` and `.rows`. The app constantly converts rows to dicts with the neat idiom `dict(zip(result.columns, row))`.
- **JSON manipulation:** the tenant PATCH endpoints use SQLite's `json_set(settings, '$.primary_color', ?)` to update a single key inside the JSON string without reading-modifying-writing the whole blob — a genuinely elegant use of SQLite's JSON1 functions.
- **Scale ceiling:** the free tier is 9 GB storage / 1 B row-reads per month. At ~100 bytes per message and sub-KB per chunk, that supports millions of messages and hundreds of clients — the binding constraint is reads, not bytes.

4. FastAPI Application

The backend is a single FastAPI app (`backend/app/main.py`) that wires together configuration, a database lifecycle, two middleware layers, and three routers — `widget` (public chat/config), `api/tenants` (tenant dashboard), and `admin` (operator-only).

4.1 Configuration — `app/config.py`

Settings are loaded by `pydantic-settings`, which reads environment variables and validates them at import time. Missing required vars raise immediately — a fail-fast design that catches misconfigured deployments at boot instead of mid-request.

Setting	Env var (alias)	Default	Purpose
`turso_database_url`	`TURSO_DATABASE_URL`	— (required)	libSQL endpoint
`turso_auth_token`	`TURSO_AUTH_TOKEN`	— (required)	DB auth
`clerk_publishable_key`	`CLERK_PUBLISHABLE_KEY`	—	JWT audience for admin verify
`clerk_secret_key`	`CLERK_SECRET_KEY`	—	JWT signature verification
`admin_email`	`ADMIN_EMAIL`	—	the only allowed admin identity
`gas_webapp_url`	`GAS_WEBAPP_URL`	—	email endpoint
`ollama_host`	`OLLAMA_HOST`	`http://localhost:11434`	local model server
`ollama_chat_model`	`OLLAMA_CHAT_MODEL`	`phi3:mini`	chat model
`ollama_embed_model`	`OLLAMA_EMBED_MODEL`	`nomic-embed-text`	embedding model
`environment` / `log_level`	`ENVIRONMENT` / `LOG_LEVEL`	`production` / `INFO`	logging
`widget_cors_origins`	`WIDGET_CORS_ORIGINS`	`*`	CORS allowlist (comma-separated)
`widget_rate_limit` / `api_rate_limit`	`WIDGET_RATE_LIMIT` / `API_RATE_LIMIT`	30 / 60	documented knobs (enforced by nginx, Chapter 2.3)

Note the pattern: `Field(..., alias="TURSO_DATABASE_URL")` makes the env var name the canonical alias, and `extra = "ignore"` tolerates stray env vars. For fields without an explicit alias, pydantic-settings still reads the matching UPPERCASE env var automatically (`OLLAMA_CHAT_MODEL` , `ENVIRONMENT` , …). A local `.env` file is supported for development.

4.2 Lifecycle and middleware stack

`main.py` composes the app in this order:

```
app = FastAPI(title="Multi-Tenant AI Chatbot API", version="1.0.0", lifespan=lifespan)
app.add_middleware(CORSMiddleware, ...)   # outermost
app.add_middleware(TenantMiddleware)      # inner: runs after CORS, before routes
app.include_router(widget.router)
app.include_router(tenant.router)
app.include_router(admin.router)
```

The `lifespan` comes from `database.py`: on startup it opens the async libSQL connection and probes the vec extension; on shutdown it closes both clients. Because startup is async, the first request never races a half-open pool — FastAPI awaits lifespan completion before serving traffic.

TenantMiddleware — how a slug becomes a tenant

```
def _extract_tenant_slug(self, request):
    path = request.url.path
    # 1. Widget endpoints: /widget/{endpoint}/{tenant_slug}
    m = re.match(r'^/widget/[^/]+/([^/]+)', path)
    if m: return m.group(1)
    # 2. API/admin endpoints: X-Tenant-Slug header
    if path.startswith('/api/') or path.startswith('/admin/'):
        h = request.headers.get('X-Tenant-Slug')
        if h: return h
        # 3. Subdomain: tenant.yourdomain.com (ignoring reserved names)
        host = request.headers.get('host', '')
        if '.' in host:
            sub = host.split('.')[0]
            if sub not in ['www', 'admin', 'api', 'widget', 'localhost']:
                return sub
    return None
```

If a slug is found, the middleware loads the tenant row into `request.state.tenant`. Two dependency functions then hand it to route handlers:

- `get_current_tenant` → 404 if absent (used by all `/api/tenants/me/*` routes).
- `get_tenant_slug` → 400 if absent.

This is the dependency-injection pattern throughout: routes declare `tenant: dict = Depends(get_current_tenant)` and FastAPI resolves it before the handler runs.

4.3 CORS

`CORSMiddleware` allows origins from `widget_cors_origins` (default `*`), all methods and headers. `allow_credentials` is only enabled when explicit origins are configured — browsers reject the wildcard + credentials combination, and none of the APIs use cookies anyway (auth is a bearer token, 4.6). The widget's fetch calls are cross-origin by design (pages.dev → hf.space); the real abuse protection is nginx rate limiting plus the fact that chat endpoints are public. Admin and tenant APIs require a signed Clerk JWT, so permissive CORS doesn't weaken them.

4.4 Endpoint catalog (30 routes)

Method	Path	Router	Auth	Purpose
GET	`/`	root	—	API name/version banner
GET	`/health`	root	—	liveness probe alias (Docker/nginx/keep-alive)
GET	`/widget/config/{slug}`	widget	—	colors, greeting, bot name, business hours for the widget
POST	`/widget/chat/{slug}`	widget	—	streaming chat (SSE)
GET	`/widget/availability/{slug}`	widget	—	available slots for a date range
POST	`/widget/appointments/{slug}`	widget	—	direct booking with email + conflict check
GET	`/widget/health`	widget	—	liveness probe (same payload as `/health`)
GET	`/widget/history/{slug}`	widget	—	message history for a conversation (verifies the conversation belongs to the tenant)
GET	`/api/tenants/me`	tenant	admin JWT + tenant ctx	own tenant profile
PATCH	`/api/tenants/me`	tenant	admin JWT + tenant ctx	update name/colors/greeting/hours/slot config
POST	`/api/tenants/me/knowledge`	tenant	admin JWT + tenant ctx	upload PDF/TXT/MD (multipart)
POST	`/api/tenants/me/knowledge/text`	tenant	admin JWT + tenant ctx	upload raw text or FAQ JSON
GET	`/api/tenants/me/knowledge`	tenant	admin JWT + tenant ctx	list sources + chunk counts
DELETE	`/api/tenants/me/knowledge/{source_id}`	tenant	admin JWT + tenant ctx	delete a source's chunks (+ vector rows)

GET	`/api/tenants/me/conversations`	tenant	admin JWT + tenant ctx	list with message_count + last_message
GET	`/api/tenants/me/conversations/{id}`	tenant	admin JWT + tenant ctx	full thread (verifies ownership!)
GET	`/api/tenants/me/appointments`	tenant	admin JWT + tenant ctx	list, optional status filter
GET	`/api/tenants/me/analytics`	tenant	admin JWT + tenant ctx	counts + resolution rate
GET	`/admin/tenants`	admin	Clerk JWT	paginated tenant list with search/plan filter
POST	`/admin/tenants`	admin	Clerk JWT	create tenant (slug uniqueness enforced)
GET	`/admin/tenants/{id}`	admin	Clerk JWT	tenant detail
PATCH	`/admin/tenants/{id}`	admin	Clerk JWT	update (same fields as tenant /me)
DELETE	`/admin/tenants/{id}`	admin	Clerk JWT	soft delete (`plan='deleted'`)
POST	`/admin/tenants/{id}/impersonate`	admin	Clerk JWT	returns slug for the frontend to "log in as"
GET	`/admin/tenants/{id}/conversations`	admin	Clerk JWT	conversations for a tenant (message count + last message + user email)
GET	`/admin/tenants/{id}/appointments`	admin	Clerk JWT	appointments for a tenant (optional status filter, user email joined)
POST	`/admin/tenants/{id}/knowledge`	admin	Clerk JWT	upload knowledge file for a tenant (multipart)
POST	`/admin/tenants/{id}/knowledge/text`	admin	Clerk JWT	upload text/FAQ for a tenant
GET	`/admin/tenants/{id}/knowledge`	admin	Clerk JWT	list knowledge sources for a tenant
DELETE	`/admin/tenants/{id}/knowledge/{source_id}`	admin	Clerk JWT	delete a tenant's knowledge source

GET	`/admin/analytics`	admin	Clerk JWT	platform-wide counts + top tenants
GET	`/admin/usage`	admin	Clerk JWT	raw usage_logs with filters

4.5 The chat endpoint, dissected

This is the heart of the product. `POST /widget/chat/{slug}`:

1. **Resolve tenant** by slug (`plan != 'deleted'`), 404 otherwise.
2. **Get-or-create conversation:** if the widget didn't send `conversation_id` , generate one and insert a `conversations` row with `end_user_id = session_id` .
3. **Persist the user message** immediately (role `'user'`) — so a crash mid-stream never loses the visitor's question.
4. **Classify intent** via `intent_service.classify()` (Chapter 5.2) and log a `usage_logs` row with the intent.
5. **Branch on intent** inside an async generator that yields SSE frames:

```
if intent == 'general_query':
    chunks = await rag_service.search(tenant_id, message)
    if chunks:
        async for piece in llm_service.synthesize_answer(message, [c['content'] for c in chunks]):
            yield f"data: {json.dumps({'type': 'content', 'content': piece})}\n\n"
    else:
        no_info = "I don't have that information in my knowledge base..."
        yield f"data: {json.dumps({'type': 'content', 'content': no_info})}\n\n"

elif intent == 'book_appointment':
    details = await intent_service.extract_booking_details(message)
    if details.get('preferred_date') and details.get('preferred_time'):
        # build naive dt, localize with pytz, convert to epoch
        slots = await appointment_service.get_availability(tenant_id, date, date, tz)
        if any(s['start_time'] == start_ts and s['available'] for s in slots):
            appt = await appointment_service.book_appointment(...)   # conflict check inside
            await email_service.send_appointment_confirmation(...)
            yield content event "Great! I've booked..."
        else:
            yield content event + yield slots event   # show alternatives
    else:
        yield content event "What date and time would you prefer?"
```

Frames are JSON objects with a `type` field: `content` (a text token), `slots` (clickable times for the widget), or `done` (final metadata + conversation_id). After the branch, the full assistant response is persisted with the `intent` attached, the conversation's `updated_at` bumps, and the generator yields `done` .

> **IMPORT NOTE.** `widget.py` imports `intent_service` / `appointment_service` from `app.services.appointments` . (An earlier revision imported them from a nonexistent `app.services.intent` module, which crashed the app at startup with `ModuleNotFoundError` — that line is fixed.)

4.6 Admin auth — Clerk JWT verification

```
async def verify_admin(credentials: HTTPAuthorizationCredentials = Depends(security)) -> str:
    header = jwt.get_unverified_header(credentials.credentials)
    key = next(k for k in _get_jwks() if k['kid'] == header['kid'])   # Clerk's JWKS public keys
    payload = jwt.decode(credentials.credentials, jwk.construct(key),
                         algorithms=["RS256"], options={"verify_aud": False})
    email = payload.get('email') or payload.get('email_address')
    if email != settings.admin_email:
        raise HTTPException(status_code=403, detail="Not authorized")
    return email
```

The security model is deliberately single-operator: the browser gets a Clerk JWT after signing in; every `/admin/*` route depends on `verify_admin`. The token must (a) verify against Clerk's **JWKS public keys** (fetched from `https://<frontend-api>/.well-known/jwks.json`, derived from the publishable key; override the URL with `CLERK_JWKS_URL`), (b) be RS256, and (c) carry *your* email. 401 for bad/absent tokens, 403 for a valid token from the wrong account.

> **SECURITY POSTURE.** (1) The tenant-management routes (`/api/tenants/me/*`) require a verified admin JWT *and* select the tenant via the `X-Tenant-Slug` header/subdomain — the earlier header-only "authentication" (anyone could spoof a slug) is closed. (2) The chat/widget routes are public by design and rely on nginx rate limiting — fine for a public bot. (3) nginx no longer exposes an `/ollama/` proxy — Ollama is only reachable inside the container. (4) The Apps Script web app is deployed as "Anyone" — its `/exec` URL is the only secret; don't publish it, because anyone with the URL can send email as you (up to the 100/day quota).

4.7 Validation, error handling, and response shapes

- **Pydantic schemas** (`schemas.py`) enforce contracts at the boundary: `ChatMessageRequest.message` must be 1–4000 chars; `TenantCreate.slug` must match `^[a-z0-9-]+$` ; `primary_color` must match `^#[0-9A-Fa-f]{6}$` ; `BookAppointmentRequest.email` is an `EmailStr` validated by email-validator. Invalid input → automatic 422 with a structured error body — no manual checks needed.
- **HTTPException** for expected failures: 404 (tenant/conversation not found), 400 (duplicate slug, unsupported file type, invalid FAQ JSON), 403/401 (admin auth).
- **LLM resilience:** `LLMService` catches Ollama errors and returns an error string (streaming path yields it as a frame); `embed()` falls back to a zero vector on failure; `extract_booking_details` returns a safe all-null dict if JSON parsing fails.
- **Response models:** most routes declare `response_model=...` , so FastAPI validates outgoing payloads too — a field rename in the schema fails loudly instead of silently breaking the widget.

5. AI Pipeline Internals

The "brain" is Ollama serving two models inside the Space: `phi3:mini` (a 3.8B-parameter instruction-tuned model from Microsoft, ~2.2 GB) for all text generation, and `nomic-embed-text` (a 137M-parameter embedding model, 768 dimensions) for vector search. `llm.py` wraps both behind an `LLMService` singleton that every other service imports.

5.1 The Ollama client

```
class LLMService:
    def __init__(self):
        self.client = ollama.AsyncClient(host=settings.ollama_host)   # http://localhost:11434
        self.chat_model = settings.ollama_chat_model                  # phi3:mini
        self.embed_model = settings.ollama_embed_model                # nomic-embed-text
```

Three primitives are used everywhere:

- `chat(messages, stream=True|False, temperature, max_tokens)` — the generator wrapper. In streaming mode it yields `message.content` pieces as they arrive; in non-streaming mode it returns the full string. Temperature and `num_predict` are passed through as Ollama `options`.
- `embed(texts)` — loop over a list, call `client.embeddings()`, collect vectors; on failure, append a 768-float zero vector so the pipeline never crashes on one bad chunk.
- `embed_single(text)` — one query embedding for RAG.

The error contract: `chat()` catches exceptions and returns `"LLM error: ..."` as the response (streaming or not) rather than raising. Downstream services therefore always get a string; the user-facing side effect is a slightly robotic answer, never a 500.

5.2 Intent classification — the router

Every incoming message is first classified into one of six buckets. The prompt is a **few-shot classifier**: the model sees labeled examples and must output exactly one label. Temperature 0.1 makes it deterministic.

```
Classify the user's intent into ONE category:
- book_appointment: Wants to schedule a meeting
- cancel_appointment: Wants to cancel or reschedule
- check_availability: Asks about open slots
- general_query: Information question
- transfer_human: Explicitly asks for human
- unclear: Ambiguous or off-topic

Examples:
User: "Book a meeting for Tuesday 2pm" -> book_appointment
User: "Cancel my appointment" -> cancel_appointment
User: "What times are open Friday?" -> check_availability
User: "What's your refund policy?" -> general_query
User: "Talk to a person" -> transfer_human
User: "Hello" -> unclear

User: "{text}"
Intent:
```

The response is matched leniently — `if valid in intent: return valid` — so `"book_appointment"`, `"book_appointment."`, or even a wrapped answer still lands in the right bucket. Anything unmatched falls through to `unclear`, which the widget answers with a polite re-prompt.

Intent	Example user messages	Backend behavior
`book_appointment`	"Book Tuesday at 2pm", "I need a consultation"	Extract details → check slot → book → email
`cancel_appointment`	"Cancel my appointment"	Ask for details (full cancel flow is not wired to the widget yet)
`check_availability`	"What times are open Friday?"	Generate slots (default: next 7 days) and stream them as a clickable picker
`general_query`	"Do you offer refunds?"	RAG search + grounded synthesis
`transfer_human`	"Talk to a person"	Friendly handoff message (no real queue yet)
`unclear`	"asdf", "hello"	Re-prompt listing capabilities

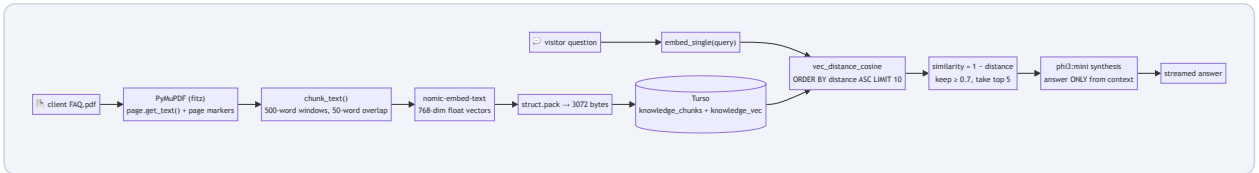
5.3 Structured extraction — booking details as JSON

Booking needs machine-readable facts, not prose. A second prompt coerces the model into emitting JSON only:

```
Extract booking details from user message. Return JSON only.

User: "{text}"

Return format:
{
  "preferred_date": "YYYY-MM-DD or null",
  "preferred_time": "HH:MM or null",
  "duration_minutes": 30,
  "title": "Appointment title or null",
  "notes": "Any notes or null"
}

If date/time not specified, use null.
```

The response is parsed with `json.loads`; on failure the service returns a safe all-null dict (falling back to the "ask for date/time" UX branch). The same pattern extracts cancellation details and business hours (Chapter 5.6).

5.4 The RAG pipeline — from PDF to grounded answers



1. **Extract** (`knowledge.py → process_pdf`): PyMuPDF opens the bytes, iterates pages, appends `\n--- Page {n} ---\n` + extracted text. The page marker survives chunking, so answers can carry approximate page metadata (`{'page': i // 5 + 1}` — a rough heuristic, one page per ~5 chunks).
2. **Chunk** (`chunk_text`): split on whitespace, slide a 500-word window with a 50-word overlap — step size 450. Overlap prevents a sentence from being cut in half at a boundary and losing meaning.
3. **Embed**: all chunks are embedded in one batch (`llm_service.embed(texts)`) — on CPU this is the slowest step of onboarding (a 20-page PDF ≈ tens of seconds).
4. **Store**: each chunk row + its packed vector row, keyed `chunk_id` (source_id + index). `INSERT OR REPLACE` makes re-uploads idempotent.
5. **Search**: query embedding → `vec_distance_cosine` → order ascending → `LIMIT top_k * 2` (fetch 10, filter to 5). The extra fetch matters because the threshold filter can discard results.
6. **Filter**: `similarity = 1 - distance` ; drop below 0.7 — the trust floor that stops the bot from answering from irrelevant chunks. (If nothing passes, the widget gets the honest "I don't have that information" line.)
7. **Synthesize**: top-5 contexts go into the grounding prompt (5.5) and the answer streams back.

5.5 The grounding prompt

```
Answer the user's question using ONLY the provided context.
If the answer isn't in the context, say "I don't have that information in my knowledge base."

Context:
Source 1: {chunk}
Source 2: {chunk}
...

Question: {query}

Answer:
```

Two design choices matter: *grounding* (answer only from context — reduces hallucination) and *honest refusal* (the model is told to say it doesn't know rather than invent). Temperature 0.2 keeps answers consistent; `max_tokens=800` bounds latency on CPU.

5.6 Business-hours extraction

Onboarding asks the AI to read the client's PDF and emit a structured week:

```
Extract business hours from the following text. Return JSON only.
Text: {text[:3000]}   # first 3000 chars — enough for a typical hours block

Return format:
{"monday": {"open": "09:00", "close": "17:00"}, ...,
 "saturday": {"open": null, "close": null}, ...
 "timezone": "America/New_York"}

If a day is closed, use null. If not mentioned, use null.
If timezone not mentioned, use "UTC".
```

Note the explicit instruction to use `null` for unknowns — the schema (Chapter 3) and slot generator (Chapter 6) both treat null as "closed", so a garbage-guessing model would silently produce phantom slots. The 3000-char cap also keeps

the prompt cheap and fast.

5.7 Context window & token budget management

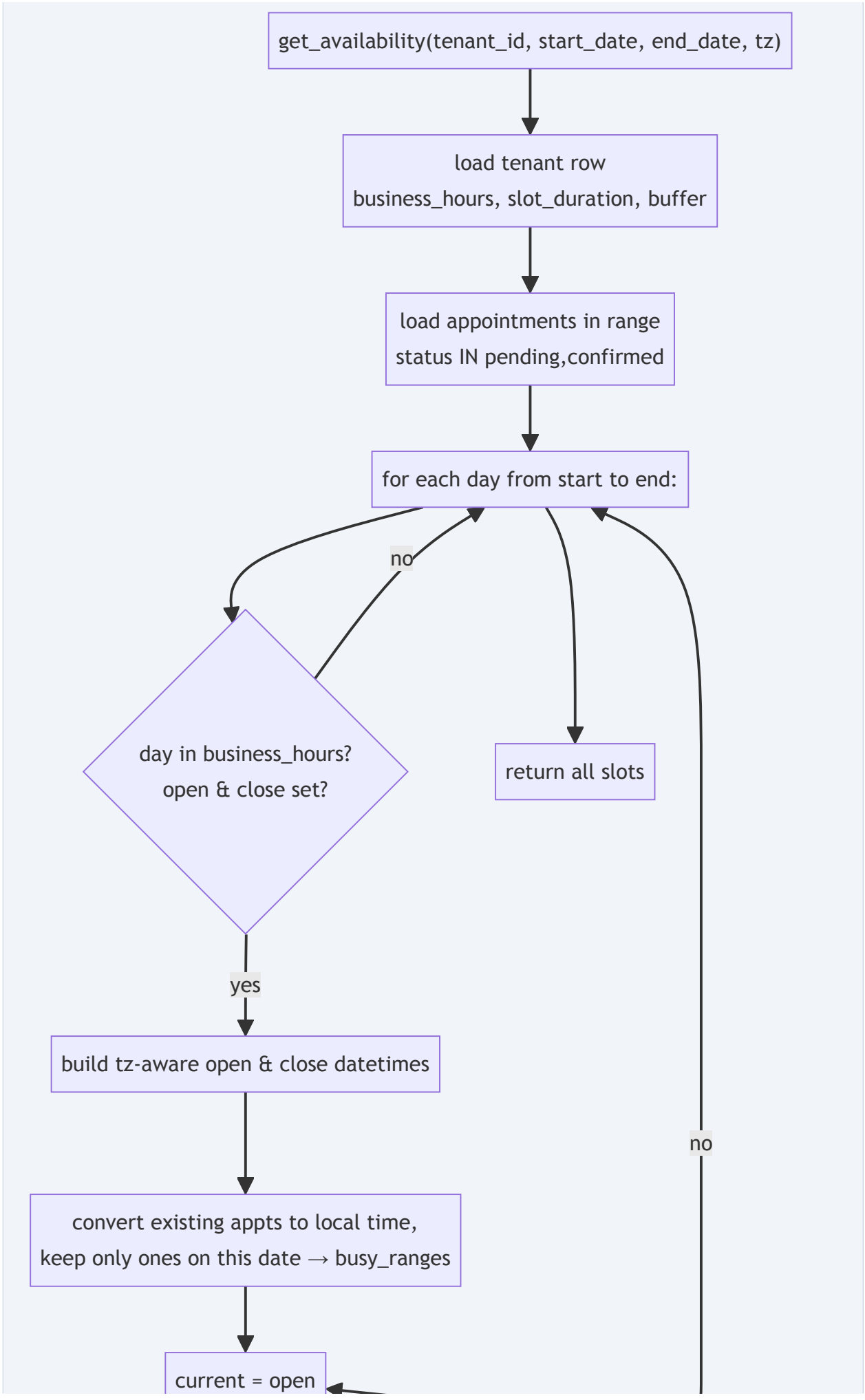
Task	temperature	max_tokens	Notes
Intent classification	0.1	20	just a label; 20 tokens is plenty and fast
Booking/cancel extraction	0.1	150–200	JSON only
Business-hours extraction	0.1	300	text capped at 3000 chars
RAG synthesis	0.2	800	5 chunks ≈ 2500 tokens of context; 800 output tokens ≈ a solid paragraph
Default chat	0.3	1000	the generic `chat()` default

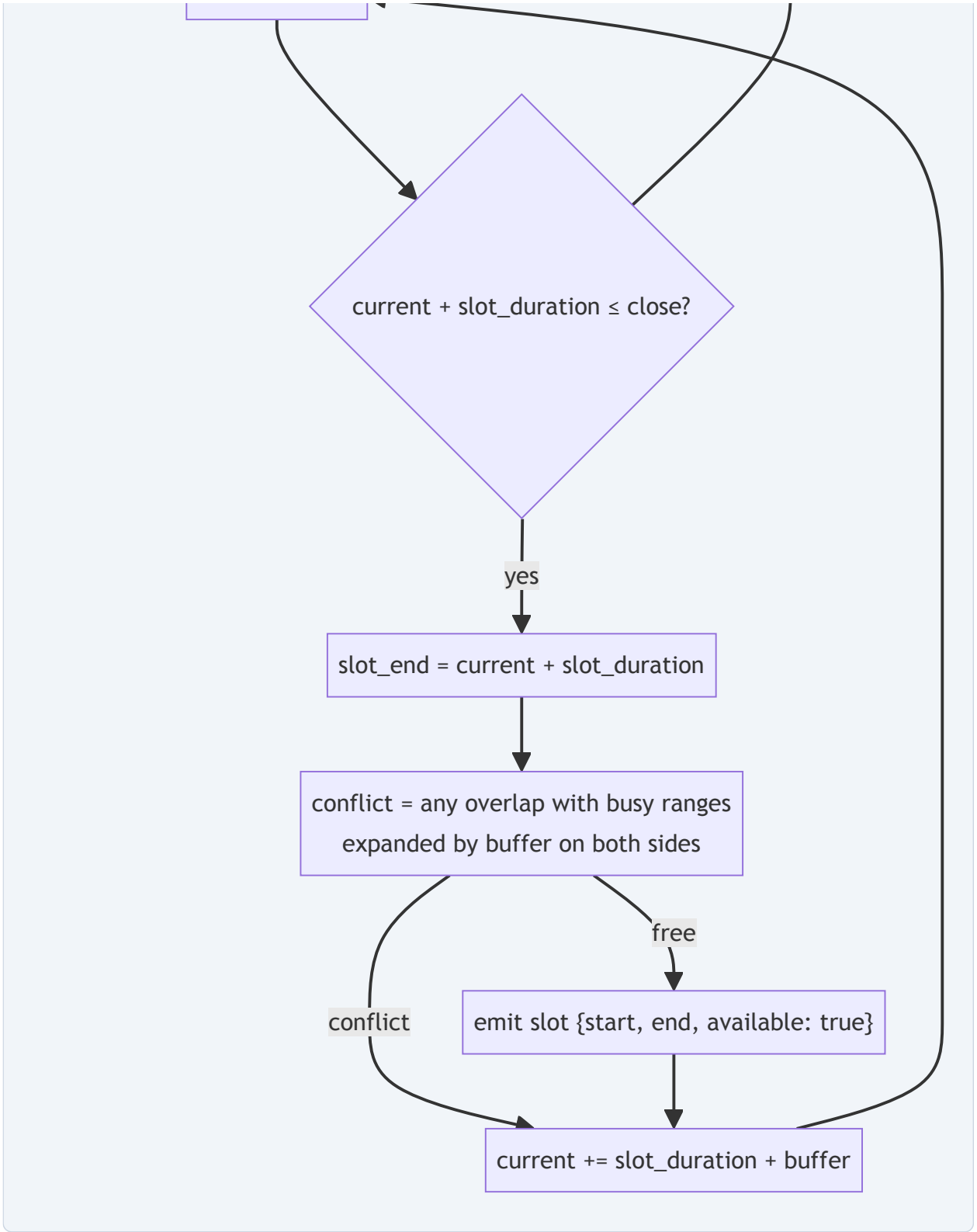
Phi-3-mini's context window is 4K tokens. With ~2.5K tokens of RAG context + 800 output tokens + prompt overhead, a single synthesis call sits near but under the limit. This is deliberate: chunk size 500 words (~650 tokens) × 5 chunks stays within budget. If `top_k` or chunk size grows, watch for context overflow (Ollama errors) — the Chapter 13 scaling notes cover the fix (smaller chunks or a larger model).

6. Appointment Engine

The appointment engine (`app/services/appointments.py`) turns three tenant settings — `business_hours` (JSON), `slot_duration` (default 30 min), `buffer_minutes` (default 15) — plus a timezone into a list of bookable Unix-timestamp ranges. It is deliberately pure Python with `pytz` : no SQL date functions, no tz magic in the database.

6.1 The availability algorithm


get_availability(tenant_id, start_date, end_date, tz)
load tenant row
business_hours, slot_duration, buffer
load appointments in range
status IN pending,confirmed
for each day from start to end:
day in business_hours?
open & close set?
no
return all slots
yes
build tz-aware open & close datetimes
convert existing appts to local time,
keep only ones on this date → busy_ranges
current = open
no



In pseudocode, the core day-generation loop:

```
for each day in [start_date, end_date):
    hours = business_hours[day_name]           # e.g. {"open": "09:00", "close": "17:00"}
    if not hours or not hours.open or not hours.close: skip  # closed day → no slots

    open_dt  = localize(date, hours.open)      # tz-aware
    close_dt = localize(date, hours.close)

    busy = [(a.start, a.end) for a in appointments if a is on this day]  # in local tz

    current = open_dt
    while current + slot_duration <= close_dt:
        slot_end = current + slot_duration
        if no overlap(current, slot_end, busy, buffer):
            emit {start: current.ts, end: slot_end.ts, available: true}
        current += slot_duration + buffer       # step INCLUDES the buffer gap
```

Buffer and overlap math — a worked example

Suppose `slot_duration = 30`, `buffer = 15`, and there's an existing booking 13:00–13:30 (local). Its *protected range* is 12:45–13:45 once the buffer pads both sides:

```
busy_start − buffer         busy_start  busy_end      busy_end + buffer
     12:45                     13:00       13:30            13:45
       |-------------------------|-----------|----------------|
                                 ▲ protected zone ▲

conflict if:  current < busy_end + buffer   AND   slot_end > busy_start − buffer
            (12:45,13:45)  vs  any candidate slot overlapping that window
```

Because every candidate step is `slot_duration + buffer` (45 minutes from 09:00), the actual candidate starts in the example are 12:00, 12:45, 13:30, 14:15, … — not every 30 minutes. Walk them through the conflict rule:

```
12:00–12:30  free   (slot ends 12:30, before the protected zone starts)
12:45–13:15  busy   (overlaps 12:45–13:45 — ends 13:15 > 12:45)
13:30–14:00  busy   (starts inside the protected zone)
14:15–14:45  free   (starts after 13:45 — the first slot after the booking)
```

So the first bookable slot after the 13:00–13:30 booking is **14:15–14:45**. The 45-minute step is why the generator never even *considers* a 13:00 or 13:15 start — buffer spacing is baked into the grid, not checked after the fact.

Conflict detection at booking time

Slot generation is advisory; the authoritative check happens in `book_appointment()`, which re-queries the database atomically enough for this scale:

```
SELECT id FROM appointments
WHERE tenant_id = ? AND status IN ('pending','confirmed')
  AND start_time < ? AND end_time > ?     -- (end_time, start_time) of the new booking
```

If any row matches, the overlap is real (not buffer-adjusted — the slot generator already enforced buffers, so the booking check is the hard no-overlap guarantee) and the booking raises `ValueError`, which the chat endpoint catches and converts into the "that time slot is no longer available" message.

6.2 Timezone handling with pytz

- **Storage:** `appointments.start_time/end_time` are epoch seconds (UTC) — unambiguous, sortable, timezone-free.
- **Day boundaries:** `get_availability` localizes the `YYYY-MM-DD` strings into the tenant's tz (`tz.localize(datetime.strptime(...))`) and adds 1 day for the exclusive end — so a request for 2026-09-08 in `America/New_York` correctly spans 04:00 UTC Sep 8 → 04:00 UTC Sep 9.
- **Busy-range comparison:** existing appointment epochs are converted to the same local tz (`datetime.fromtimestamp(ts, tz=utc).astimezone(tz)`) and filtered by calendar date, so appointments from other days never interfere.
- **DST:** using `pytz.timezone().localize()` (not `replace(tzinfo=)`) applies DST rules correctly — a 09:00 open on a DST-transition day still maps to the right UTC instant.

6.3 Defaults and edge cases

Case	Behavior
No `business_hours` set	Fallback default: Mon–Fri 09:00–17:00, weekend closed (`get_business_hours`)
Day missing from JSON / null open	Treated as closed → zero slots that day
Existing appointments *anywhere* in range	Loaded once (status pending/confirmed) and converted per-day — one DB query for the whole window
Booked slot when free at generation but taken at booking	Second visitor's booking conflicts → friendly fallback message + fresh slot list
End-of-day slot	Loop condition `current + duration <= close` — a 17:00–17:30 slot needs 17:30 ≤ 17:00 to be true, so it's correctly excluded

> **RECURRENCE.** The engine has *no recurrence patterns* — every appointment is a one-off. "Every Monday at 9am" style rules would need a new `recurrence` column + expansion logic; the Chapter 13 roadmap lists it as the natural next feature for clinics and tutors.

6.4 Booking flow (direct endpoint + chat)

1. Widget/chat extracts or receives `start_time` + `end_time` (epoch).
2. `book_appointment()`: conflict query → insert with status `'confirmed'` → return dict.
3. Widget endpoint also creates/updates an `end_users` row (email + name) and a `conversations` row, then sends the customer confirmation email *and* (if `notification_email` is in tenant settings) the business notification.
4. The chat path emails `customer@example.com` — a placeholder (Chapter 12 flags this); the direct booking endpoint uses the visitor's real email from the request body.

7. Email System

Email is outsourced to a **Google Apps Script web app** — a single file (`gas-email/Code.gs` , ~60 lines of code plus setup comments) deployed as "Web App, anyone can access, run as me". FastAPI POSTs JSON to its `/exec` URL; Apps Script turns it into a Gmail send; the response is a tiny JSON document. No SMTP, no send-grid, no keys beyond the URL itself.

7.1 The Apps Script endpoint

```
function doPost(e) {
  try {
    const data = JSON.parse(e.postData.contents);
    const { to, subject, html } = data;
    if (!to || !subject || !html) {
      return ContentService.createTextOutput(JSON.stringify({ error: 'Missing required fields' }))
        .setMimeType(ContentService.MimeType.JSON);
    }
    MailApp.sendEmail({ to, subject, htmlBody: html, name: 'Chatbot Appointments' });
    return ContentService.createTextOutput(JSON.stringify({ success: true, messageId: 'sent' }))
      .setMimeType(ContentService.MimeType.JSON);
  } catch (error) {
    return ContentService.createTextOutput(JSON.stringify({ error: error.toString() }))
      .setMimeType(ContentService.MimeType.JSON);
  }
}
```

Three contract details:

- **Input**: `{to, subject, html}` — validated server-side (a bare 400-style error JSON is returned, no email sent).
- **Output**: always JSON with `success` or `error` — `email.py` reads `result.get('success', False)` , so a malformed reply is treated as failure, never as success.
- **Run-as-me authorization**: emails are sent from *your* Gmail account and land in your Sent folder; the first call (running `testEmail()` in the editor) triggers Google's OAuth consent, which is why the setup checklist always includes "run testEmail once".

7.2 CORS and callers

Apps Script web apps have *no CORS headers* — a browser calling the `/exec` URL directly gets blocked. That's fine and intended: the only caller is `email_service.send_email()` in FastAPI, which uses `httpx.AsyncClient(timeout=10.0)` server-to-server. Keeping email dispatch server-side also hides the endpoint from the public widget and keeps the abuse surface (spam through your Gmail) closed.

7.3 HTML email templates

`email.py` builds two templates with inline styles (required for Gmail — it strips `<style>` blocks):

- **Customer confirmation/reminder** (`send_appointment_confirmation`): a card with the business name, greeting, and a blue-bordered details block (Date / Time / Service / Notes). `is_reminder=True` swaps the subject to "Reminder: ... tomorrow" — the hook for a reminder scheduler that isn't wired up yet.

- **Business notification** (`send_business_notification`): "New Appointment Booked" with customer name, email, date, time — sent when the tenant's `settings.notification_email` is set.

Layout is deliberately retro-safe: `max-width: 600px` , `margin: 0 auto` , flat backgrounds, `border-left: 4px solid #3b82f6` accent — renders identically in Gmail, Apple Mail, and Outlook.

7.4 Quotas

Account type	Daily send limit	Note
Personal Gmail (via GAS)	100 emails/day	the default; ~15/day is realistic for a pilot client
Google Workspace	1,500 emails/day	$6/user/mo upgrade path

`Code.gs` ships a `checkQuota()` helper (`MailApp.getRemainingDailyQuota()`) for monitoring. When the daily cap is hit, MailApp throws — the try/catch returns an error JSON, `send_email` logs "Email send error" and returns False, and the booking still succeeds (the email is a notification, not a transaction).

7.5 Reliability notes

- **No retry logic today:** one POST, 10 s timeout, boolean result. For a booking business, a dropped confirmation is annoying but not fatal (the chat confirms in-line first). The obvious hardening — a 3-attempt backoff or a queue table — is listed in Chapter 13.
- **Failure is silent at the UI:** the chat booking branch doesn't check the email result, so a Gmail quota failure won't break the conversation — a deliberate resilience choice.
- **Test path:** `scripts/test_gas_email.py --to you@example.com` POSTs a canned HTML message and asserts `success` — the standard first thing to run when "no emails" is reported (Chapter 12).

8. Widget (React + Shadow DOM)

The widget is a React 18 app compiled by Vite into one **IIFE file**, `widget.js`, that any website can load with a single `<script>` tag. It renders its own floating chat button, panel, and slot picker in a **Shadow DOM** so it can't collide with the host site's CSS. ⚠ As shipped, the build needs three one-line fixes before it actually produces that file — 8.6 verifies each one against the real Vite output.

8.1 Bootstrapping — `main.tsx`

```
const scriptTag = document.currentScript as HTMLScriptElement
const tenantSlug = scriptTag?.dataset?.tenant || 'demo'   // data-tenant="weblyft-design"

const host = document.createElement('div')
host.id = 'chatbot-widget-host'
document.body.appendChild(host)

const shadowRoot = host.attachShadow({ mode: 'open' })    // style isolation boundary
const styleSheet = document.createElement('style')
styleSheet.textContent = `/* styles injected by Vite */`
shadowRoot.appendChild(styleSheet)

const mountPoint = document.createElement('div')
mountPoint.id = 'chatbot-widget-root'
shadowRoot.appendChild(mountPoint)

const root = createRoot(mountPoint)
root.render(<ChatWidget tenantSlug={tenantSlug} />)
```

Everything the widget draws lives inside `shadowRoot`: its `<style>` rules (scoped by the shadow boundary) and the React tree. The host page's CSS simply cannot see in — `.chatbot-button` on the client's site can't restyle our button, and our `div`s can't inherit their ugly theme. (Mode `'open'` means devtools can still inspect it.)

Note that the `<style>` element is created empty — its text is supposed to come from `styles.css`. In a Vite *library* build, CSS imported normally is emitted as a separate `style.css` asset instead of being inlined, which would leave the widget unstyled. The fix in 8.6 imports the CSS as a string (`?inline`) so the bundle is genuinely self-contained.

> **SHADOW DOM CAVEATS.** (1) Fonts don't inherit through the shadow boundary — the widget uses system font stacks, which is why it looks consistent everywhere. (2) `z-index` is relative to the shadow tree; if a client's site has aggressive stacking contexts, the button can still end up underneath a sticky header. (3) If the host site sends a strict `Content-Security-Policy` without allowing `connect-src` to the Space origin, every fetch fails silently (Chapter 12 has the client-side checklist).

8.2 The three hooks

`ChatWidget.tsx` composes three custom hooks:

Hook	State	Behavior

`useConfig(tenantSlug)`	`data | loading | error`	Fetches `/widget/config/{slug}` on mount; until it resolves, the widget renders `null` (nothing visible) so the button always appears with correct colors/greeting
`useSession(tenantSlug)`	`sessionId` (stable), `conversationId`	Creates a persistent `session_{timestamp}_{random}` in `localStorage` (key per tenant). It exposes `saveConversationId` / `clearSession` for the conversation key — but nothing in `ChatWidget` calls them yet, so a returning visitor only resumes a thread once that wiring lands (8.4, 12.1)
`useChat(tenantSlug, conversationId)`	`messages, isLoading, slots, pendingAction`	The SSE engine (8.3) plus optimistic user bubbles, error bubbles, slot state, and history loading

8.3 SSE streaming — the fetch-reader loop

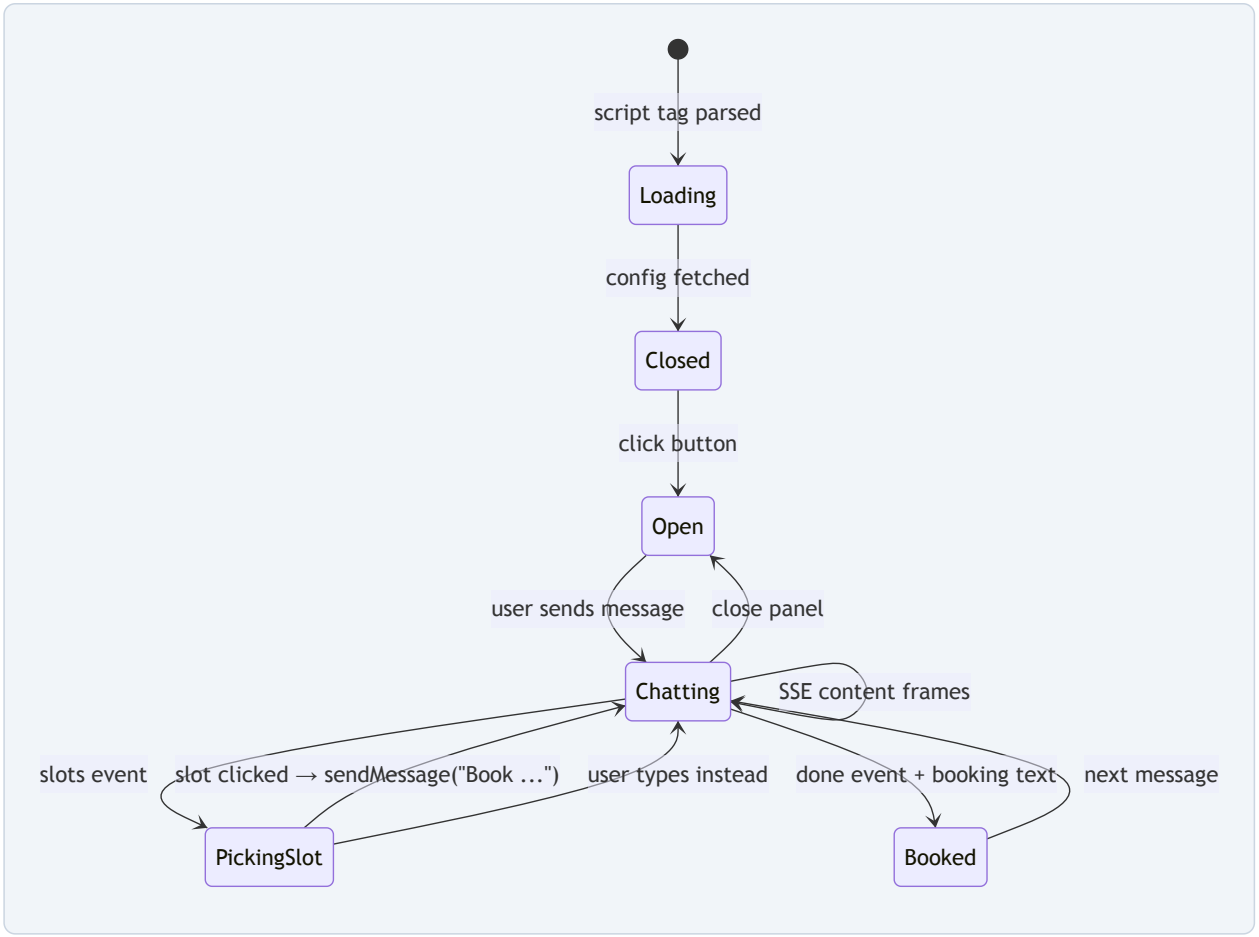
SSE over `fetch()` + `ReadableStream` (not `EventSource`) is used deliberately: `EventSource` only supports GET, but chat needs POST with a JSON body.

```
const response = await fetch(`/widget/chat/${tenantSlug}`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ message, conversation_id, session_id })
})

const reader = response.body?.getReader()
const decoder = new TextDecoder()
let buffer = ''

while (true) {
  const { done, value } = await reader.read()
  if (done) break
  buffer += decoder.decode(value, { stream: true })   // decode + KEEP partial trailing bytes
  const lines = buffer.split('\n')
  buffer = lines.pop() || ''                           // last line may be incomplete — hold it
  for (const line of lines) {
    if (line.startsWith('data: ')) {
      handleStreamData(JSON.parse(line.slice(6)))       // {type: content|slots|done, ...}
    }
  }
}
```

The buffer dance is the classic SSE gotcha: a chunk can split a `data:` line in half, so the reader keeps the trailing fragment until the next chunk completes it. `handleStreamData` then switches on `type`:

- `content` → append to the last assistant bubble (or create one) — this is what types the answer out live.
- `slots` → stash slots, set `pendingAction = 'pick_slot'` → the panel renders the `SlotPicker`.
- `done` → persist `conversation_id`, mark the last message read (clears the unread badge logic), clear pending action.

8.4 Booking state machine



Key detail: clicking a slot doesn't call a booking API — it synthesizes the message `"Book {date} at {time}"` and sends it through the same chat pipeline. Intent classification recognizes it, re-validates the slot, and books. One code path for everything is why the widget stays tiny.

> **WIRING CAVEAT — THE PICKER BELOW IS DESCRIBED AS DESIGNED.** In `ChatWidget.tsx` as shipped, the `slots`, `pendingAction`, and `selectSlot` values from `useChat` are never passed into `ChatPanel`, so `SlotPicker` never actually renders and clicking a slot is unreachable. Combined with the build fixes in 8.6, wiring those three props is the remaining step to make the booking UX work end to end (Chapter 12 catalogs it).

8.5 Components

Component	Job
`ChatButton`	Floating action button (bottom-right), brand color via CSS var, unread badge capped at "9+"
`ChatPanel`	The window: header (logo, bot name, "Online now", close), message area, slot picker, input, "Powered by AI Chatbot" branding (configurable via `show_branding`)

`MessageList` / `MessageBubble`	Rendering; welcome state shows the greeting from config; typing indicator (3 bouncing dots) while streaming
`InputBar`	Auto-resizing textarea (capped 120 px), disabled while streaming, Enter-to-send
`SlotPicker`	Groups slots by date (`toLocaleDateString`), renders each as a clickable time chip, disables unavailable ones

Styling uses CSS custom properties threaded from config: `--primary-color` and `--secondary-color` are set as inline styles on the panel/button, so each tenant's colors flow without touching component logic.

8.6 Vite lib-mode build

```
build: {
  lib: {
    entry: 'src/main.tsx',
    name: 'ChatbotWidget',
    fileName: 'widget',
    formats: ['iife']          // one self-executing file, no imports
  },
  outDir: 'dist',
  minify: 'esbuild',
  target: 'es2020',
  define: { 'process.env': {} }   // stub for any lib that sniffs NODE_ENV
}
```

IIFE means React, ReactDOM, and all components are bundled *into* one file — that's the whole point of the widget: a single `<script>` tag, no other dependencies.

> **BUILD NOTE — FIXED IN THE CURRENT REPO.** Earlier revisions had three issues that stopped the widget from building: (1) `main.tsx` imported a *default* export while `ChatWidget.tsx` only exports a *named* one; (2) Vite library builds emit CSS separately, so the shadow-root style element stayed empty; (3) the output was `dist/widget.iife.js`, not `widget.js`. All three are fixed: `main.tsx` uses `import { ChatWidget }`, inlines the styles via `import styles from './styles.css?inline'` + `styleSheet.textContent = styles`, and `vite.config.ts` forces the exact filename with `fileName: () => 'widget.js'`. `npm run build` now produces one self-contained `dist/widget.js` (≈ 489 KB raw, ≈ 151 KB gzipped), and every URL in this guide and the practical guide works as written.

8.7 CSP considerations

- **Host-site CSP:** `script-src` must allow the pages.dev origin; `connect-src` must allow the HF Space origin (for fetch/SSE); `style-src` may need `'unsafe-inline'` because the shadow root injects a `<style>` element.
- **Same-origin nuance:** because the widget fetches relative paths, serving `widget.js` from the same origin as the API (via proxy or directly from the Space) sidesteps CSP and CORS entirely — the cleanest production setup (Chapter 2.5).

9. Admin Dashboard (React + Clerk)

The dashboard is a React 18 SPA (Vite) on Cloudflare Pages, protected by **Clerk**. It's the operator's control room: tenants, per-tenant data, knowledge uploads, and platform analytics.

9.1 Clerk auth flow

Client side: `main.tsx` wraps the app in `<ClerkProvider publishableKey={import.meta.env.VITE_CLERK_PUBLISHABLE_KEY}>`. `App.tsx` gates rendering with `<SignedIn> / <SignedOut>` — signed-out users see the hosted `<SignIn />` screen; signed-in users get the full app. `useAuth()` exposes `isLoaded` / `isSignedIn` so the router doesn't flash protected pages before Clerk hydrates. (That publishable key is baked in at *build time*: set `VITE_CLERK_PUBLISHABLE_KEY` as a build-time environment variable in Cloudflare Pages — or in the workflow's `npm run build` step — or the key is empty and the sign-in screen can't render.)

Server side: every `/admin/*` call carries the Clerk JWT in `Authorization: Bearer ...`. The backend's `verify_admin` dependency (Chapter 4.6) verifies the signature against Clerk's JWKS public keys (RS256), then — the critical gate — compares the token's email to `ADMIN_EMAIL`. A valid Clerk session from anyone else gets 403.

> **WHO ATTACHES THE TOKEN?** The Clerk React SDK does not inject the session token into arbitrary `fetch` calls, so the dashboard wraps requests in `api.ts` (`apiFetch`): `App.tsx` registers a token getter from `useAuth().getToken()` once signed in, and `apiFetch` adds `Authorization: Bearer <token>` to every admin/tenant call. Tenant detail views also use admin-scoped endpoints (`/admin/tenants/{id}/conversations|appointments|knowledge`) so no `X-Tenant-Slug` header is needed.

9.2 Routing — React Router v6

```
<Layout>                                        // sidebar + header + <Outlet />
  <Routes>
    <Route path="/" element={<Navigate to="/tenants" replace />} />
    <Route path="/tenants" element={<Tenants />} />
    <Route path="/tenants/:tenantId" element={<TenantDetail />} />
    <Route path="/analytics" element={<Analytics />} />
    <Route path="/settings" element={<Settings />} />
  </Routes>
</Layout>
```

Patterns worth noting: the catch-all `Navigate replace` keeps the URL canonical; `Layout` uses `<Outlet />` (v6 nested-route rendering) with `useLocation()` to highlight the active nav link; `TenantDetail` reads `useParams().tenantId` for its data fetch.

9.3 Page-by-page

Page	What it does	API calls
`Tenants`	Card grid of clients (avatar with first letter, slug, plan badge, created date); "Add Tenant" modal (name, slug sanitized to `[a-z0-9-]`, color picker)	`GET /admin/tenants`, `POST /admin/tenants`

`TenantDetail`	5 tabs: Overview (stat cards: conversations, appointments, confirmed, pending), Conversations (table with message count + last message), Appointments (date/time/title/status/customer), Knowledge (`FileUploader`), Settings (`BusinessHoursEditor`)	`GET /admin/tenants/{id}` , plus tenant-scoped endpoints
`Analytics`	6 stat cards (tenants, active 30d, conversations, appointments, messages 7d, appointments 7d) + "Top Tenants by Conversations" table	`GET /admin/analytics`
`Settings`	placeholder card ("coming soon")	—

9.4 File upload with progress

`FileUploader` supports three source types:

- **PDF / TXT / MD:** one `FormData` per file (`file` , `source_id` = filename minus extension, `source_type`), POSTed to `/api/tenants/me/knowledge` . Multiple files upload sequentially, each reporting `✓ name uploaded` / `✗ name failed` .
- **FAQ:** a JSON array of `{question, answer}` typed into a textarea, POSTed to `/api/tenants/me/knowledge/text` with `source_type: 'faq'` ; the backend parses and stores each Q/A as its own chunk.

There's a `uploading` state (button shows "Uploading…") but no per-file progress bar — the honest improvement is a `XMLHttpRequest` `upload.onprogress` or a chunk-count callback, since PDF processing can take tens of seconds on CPU.

9.5 Business hours editor

`BusinessHoursEditor` manages local state: a `DAYS` array drives rows; each row has an open checkbox + two `<input type="time">` fields; empty open/close = day closed; plus selects for timezone (9 common zones), slot duration (15/30/45/60), and buffer (0–30). `handleSave` packages everything into `{business_hours, timezone, slot_duration, buffer_minutes}` and calls `onSave` — which in the current `TenantDetail` is a `// TODO` that only `console.log` s. Wiring it to `PATCH /api/tenants/me` is on the roadmap (Chapter 12/13).

9.6 Analytics queries

`get_platform_analytics` runs six cheap aggregate queries:

```
SELECT COUNT(*) FROM tenants WHERE plan != 'deleted';                    -- total tenants
SELECT COUNT(DISTINCT tenant_id) FROM usage_logs WHERE created_at > ?;   -- active 30d
SELECT COUNT(*) FROM conversations;                                      -- total threads
SELECT COUNT(*) FROM appointments;                                       -- total bookings
SELECT COUNT(*) FROM messages WHERE created_at > ?;                      -- msgs last 7d
-- top tenants by conversation volume:
SELECT t.name, t.slug, COUNT(c.id) AS conv_count
FROM conversations c JOIN tenants t ON c.tenant_id = t.id
GROUP BY t.id ORDER BY conv_count DESC LIMIT 10;
```

The "active tenants" metric leans on `usage_logs` (written on every chat message) — a nice illustration of why the log table exists. All queries are tenant-id-agnostic platform rollups, safe to run at this scale.

10. CLI Scripts

Five Python scripts in `scripts/` handle everything the dashboard doesn't (yet): onboarding clients, uploading knowledge, extracting hours, testing email, and seeding a demo. They all share the same bootstrap pattern:

> **WHERE DO THESE SCRIPTS RUN?** `run_migrations.py` *is* copied into the image and runs automatically on boot (supervisord). The `scripts/` folder is *not* copied into the Docker image, so to run `create_tenant.py` / `upload_knowledge.py` from the Space terminal, either add `COPY scripts ./scripts` (and `COPY backend ./backend`, since the scripts import `backend.app.*`) to the Dockerfile and redeploy, or run them from your own machine — the database is remote, so location doesn't matter. `create_tenant.py` and `test_gas_email.py` (no AI needed) run anywhere with `pip install -r backend/requirements.txt` and a `.env` containing the Turso credentials (`GAS_WEBAPP_URL` for the email test). The embedding-heavy scripts (`upload_knowledge.py`, `extract_business_hours.py`) call Ollama at `localhost:11434`, which only exists inside the Space — run those there (after the COPY) or tunnel Ollama's port locally.

```
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
# makes `backend.app.*` importable when run from anywhere:
#   python scripts/create_tenant.py  →  repo root goes on sys.path

from backend.app.database import Database
from backend.app.config import settings

db = Database()
sync_db = db.sync_connect()      # sync libSQL client — no asyncio needed for one-shot commands
```

10.1 create_tenant.py

`argparse` with `--name` (required), `--slug` (required), `--color` (default `#3B82F6`), `--greeting`. Flow: validate slug against `^[a-z0-9-]+$` → connect → reject duplicate slug (exit 1) → insert tenant with a UUID + default settings JSON → print the id/name/slug plus the ready-to-paste embed code. This mirrors `POST /admin/tenants` exactly, so scripts and API never drift.

10.2 upload_knowledge.py

Args: `--tenant`, `--file`, `--source-type` (pdf|txt|md, default pdf). Steps: look up tenant by slug (exit 1 if missing) → read file bytes → derive `source_id` by replacing dots in the filename (`faq.pdf` → `faq_pdf`) → call `knowledge_service.process_pdf` or `process_text` → print `chunks_created` and `total_chars`. Because embedding is async, the whole script is an `async def main()` run via `asyncio.run(main())` — the cleanest way to reuse the app's async services from a script.

10.3 extract_business_hours.py

Opens the PDF with PyMuPDF, concatenates per-page text (same page-marker format as the knowledge pipeline), hands the first ~3000 chars to `knowledge_service.extract_business_hours` (the LLM prompt from Chapter 5.6), then writes the result JSON into `tenants.business_hours`:

```
sync_db.execute("UPDATE tenants SET business_hours = ?, updated_at = strftime('%s','now') WHERE id = ?",
                [json.dumps(business_hours), tenant_id])
```

Failure modes handled: missing tenant/file (exit 1), empty extraction (prints "No business hours found" and returns without touching the DB).

10.4 test_gas_email.py

Args `--to` (required), `--url` (optional override). POSTs a canned HTML test message to the Apps Script endpoint with a 15 s timeout, checks `success`, prints ✓/✗, exits 1 on failure. This is the smoke test for the entire email leg.

10.5 seed_demo.py

Idempotent demo bootstrap: creates the `demo` tenant (with realistic business hours and greeting) if it doesn't exist, then loads six sample FAQ pairs through `knowledge_service.process_faq` unless the `demo-faq` source already exists. Every statement is guarded, so re-running is always safe — the same `INSERT OR REPLACE`/existence-check philosophy as the migrations.

10.6 Patterns worth copying

- **Fail fast with exit codes:** every pre-condition (tenant exists, file exists, slug valid) is checked before any write; errors go to `stderr`/exit 1 so CI or a human can rely on the status.
- **Print the next action:** each script ends by telling you exactly what to paste next (embed code, verification query) — onboarding by copy-paste.
- **Sync client for scripts, async for API:** one-shot commands don't need an event loop unless they call services that await (`asyncio.run` only where required).

11. Monitoring & Operations

With a $0 budget, monitoring is a patchwork of free tools plus deliberate logging. Here's the full picture.

11.1 Uptime checks

Layer	Mechanism	Interval	What it catches
Docker HEALTHCHECK	`curl /health` inside the container (served by FastAPI — 2.1)	every 30 s	FastAPI down / OOM inside the Space
GitHub Actions keep-alive	external curl from GitHub's runners to `https://USER-SPACE.hf.space/health` (returns 200 — 2.1)	every 30 min	Space sleeping; external reachability (DNS, TLS, HF routing)
Uptime Kuma	recommended self-hosted (or cloud instance) monitor on `https://chatbot-api.YOUR-SUBDOMAIN.workers.dev/health`	every 60 s	real-user-path failures + notification via Telegram/email/webhook

The three layers complement each other: HEALTHCHECK proves the process is alive *inside*, keep-alive keeps the free Space from sleeping, Kuma gives *you* the phone notification when the site's bot stops answering.

11.2 HF Space sleep behavior

Free Spaces suspend after ~48 h of zero traffic. The keep-alive ping resets the timer, so the bot is effectively always warm. If it *does* sleep (e.g. GitHub Actions was disabled), the first visitor request takes ~30 s while the container boots and Ollama reloads models into RAM. Symptoms in the widget: the button loads instantly (static file) but the first chat message spins for ~30 s before tokens appear. Fix: re-enable keep-alive, or accept the cold start.

11.3 Turso usage & alerts

- **Dashboard:** Turso's UI shows storage (9 GB free) and row reads (1 B/month free) live; watch the reads graph — RAG searches read many rows per message (10 candidate chunks × 2 tables + tenant rows).
- **Alerts:** Turso free tier sends no proactive alerts; add a manual monthly check (calendar reminder) or a cheap Grafana/healthcheck-style script that queries `SELECT COUNT(*) FROM messages` and pages you at a threshold.
- **Degradation warning signs:** chat latency creeping up (index scan growth), 429-ish read errors in `fastapi_err.log`, or Turso dashboard red bars near quota.

11.4 Gmail quota monitoring

100 emails/day on personal Gmail. Monitor by running `checkQuota()` in the Apps Script editor, or add a scheduled trigger. Realistic usage (15 emails/day at pilot scale) leaves an 85% headroom buffer; hit 80 emails and it's time to talk Workspace ($6/user/mo, 1,500/day).

11.5 Where the logs live

Log	Location	Useful for
supervisord master	`/var/log/supervisor/supervisord.log`	which child started/restarted and when
Ollama	`/var/log/supervisor/ollama.log` (+ `_err`)	model load failures, OOM, connection refused
FastAPI stdout/stderr	`/var/log/supervisor/fastapi.log` / `fastapi_err.log`	the app's `print()` diagnostics: "LLM error", "Email send error", "Could not load vec extension"
nginx	`/var/log/nginx/access.log` / `error.log`	rate-limit rejections (503s), 404s, SSE disconnects

All are reachable from the Space terminal (Settings → Open in terminal → `tail -f /var/log/supervisor/fastapi_err.log`).

11.6 Cost tracking

The platform costs $0/month in normal operation. The only variable inputs worth tracking in a spreadsheet: **time** (your hours × rate), **Turso reads** (usage grows with traffic), **Gmail sends**, and the **first paid upgrade** triggers (Chapter 13). A simple monthly ledger: revenue per client vs. hours spent per client — that's the signal for when to raise prices or hire a VA (see the practical guide, Chapter 8).

12. Troubleshooting Encyclopedia

This chapter is organized by component. For each issue: symptom → root cause → fix. A decision tree at the end routes you when you're not sure where to start.

12.1 Widget

Symptom	Root cause	Fix
No floating button at all	`data-tenant` misspelled / script URL wrong / JS error	Check console (F12): network tab shows the script request; verify slug matches exactly; verify the widget file URL resolves (the build emits `dist/widget.js` — see 8.6)
Widget deploy fails: `"default" is not exported by "src/ChatWidget.tsx"`	named-vs-default export mismatch (fixed in the current repo — `main.tsx` uses the named import)	Rebuild and redeploy; if you're on an old revision, apply the 8.6 fixes
Widget loads but is completely unstyled	Vite library build emitted `style.css` separately; the shadow-root style element was empty (fixed in the current repo via `styles.css?inline`)	Rebuild and redeploy; if you're on an old revision, apply the 8.6 fixes
Button shows, chat panel never opens with correct colors	`GET /widget/config/{slug}` fails (404 tenant, CORS, or network)	Curl the config URL; confirm tenant exists (`plan != 'deleted'`); check CORS on the Space
"Failed to fetch" on first message	Relative `/widget/chat/...` URL hitting the wrong origin; host-site CSP `connect-src` blocks the Space; CORS	Serve widget.js same-origin as API or add proxy; client adds Space origin to CSP; verify nginx rate-limit isn't 503-ing
Message sent, spinner forever, no text	SSE stream stalls — nginx buffering re-enabled, or 300 s timeout hit, or Ollama generating slowly on cold start	Check `proxy_buffering off` on `/widget/chat/`; check fastapi log for LLM errors; wait out cold start (~30 s)
Styles look broken / clash with site	Shadow DOM not applied (old cached build?)	Hard-refresh; re-deploy widget; verify `main.tsx` attachShadow path ran (console)
Conversation resets on refresh	localStorage key `chatbot_conversation_{slug}` missing/cleared (incognito, cleared storage)	The widget now persists the conversation id (`saveConversationId`) when the `done` frame arrives; a refresh resumes the same thread unless storage was cleared
Clicking ✕ on the slot picker errors	Close button called `onSelect(null)`, which crashed in `selectSlot` (fixed — `selectSlot(null)` now just dismisses the picker)	Rebuild and redeploy the widget; on an old revision, guard `selectSlot` against `null`

12.2 Backend / Space

Symptom	Root cause	Fix
"Space healthy" URL returns 404 although the app runs	Container still booting (first deploy), or an old image without the `/health` alias	Wait for the deploy to finish; both `/health` and `/widget/health` are served by current code (2.1)
Space terminal: `python backend/run_migrations.py` → "can't open file"	`scripts/` is not in the Docker image; `run_migrations.py` is copied to `/app` (2.1), so use `python run_migrations.py` from `/app`	Run `python run_migrations.py` (or from your machine against the remote DB); migrations also auto-run on container boot
ImportError / ModuleNotFoundError on `app.services.intent`	Old revision — `widget.py` imported from a nonexistent module (fixed; it now imports from `app.services.appointments`)	Pull the current code and redeploy
Ollama "connection refused" from FastAPI	Ollama not up when FastAPI started; model not pulled	supervisord should restart it; verify `ollama list` in terminal; `ollama pull phi3:mini nomic-embed-text`
Requests rejected with 503 + "limit_req" in nginx log	Rate limit burst exceeded (30 r/s widget, 60 r/s API)	Legit traffic spike or a loop in the widget; raise burst/rate in nginx.conf and redeploy
All routes 500 immediately after deploy	Missing env var (pydantic-settings fails at import) or DB unreachable	Check Space secrets; `python -c "from app.config import settings"` in terminal; test `TURSO_DATABASE_URL` reachability
Email endpoint errors "Could not load vec extension"	sqlite-vec not loadable in this libSQL build	Non-fatal (warn-only) by design; ensure the migration ran against the same DB the app connects to

12.3 Database

Symptom	Root cause	Fix
"Tenant not found" for a tenant you just created	Slug mismatch (case/spacing) or row created with `plan='deleted'`	`SELECT slug, plan FROM tenants;` in Turso; re-run create script with exact slug
Tables missing (every query errors)	Migration never ran against *this* database (e.g. DB swapped after first boot)	The container auto-migrates on boot; to force it: `python run_migrations.py` — idempotent, safe to re-run (or run it on any machine with the Turso URL/token in `.env`, since the DB is remote)
Vector search returns nothing for obvious matches	Embeddings stored as zero-vectors (embed failure fallback) or similarity threshold 0.7 too strict for short queries	Re-upload knowledge after checking Ollama embed model is pulled; optionally lower threshold in `rag.py`
Double bookings possible	Two requests racing between slot check and INSERT	Current code is best-effort at this scale; true fix: unique constraint on (tenant_id, start_time) or a transaction (Chapter 13)

Reads quota climbing fast	RAG searches ×2 tables ×10 candidates per message	Reduce `top_k*2` fetch, cache query embeddings, or paginate dashboards (all Chapter 13 options)

12.4 AI answers

Symptom	Root cause	Fix
Bot answers "I don't have that information" for known facts	Chunks below 0.7 similarity; content not uploaded; embedding mismatch	Re-upload; verify chunks exist (`SELECT COUNT(*) FROM knowledge_chunks WHERE tenant_id=...`); check the embedding model is nomic-embed-text everywhere
Answers are right but wordy/garbled	Temperature too high for the task	Synthesis is 0.2 by design; if still messy, tighten the grounding prompt or reduce context to 3 chunks
Wrong intent (e.g. "book" triggers general_query)	Classifier prompt edge case / ambiguous phrasing	Add the failing phrase as a few-shot example in `classify_intent` and redeploy
Slow first answer (~10–30 s)	CPU inference + cold model load	Normal for free tier; keep-alive prevents the 30 s wake; consider `num_predict` caps (already 800)
"LLM error: ..." text in chat	Ollama down / OOM / model missing	Check ollama.log; `ollama list` ; restart Space from HF dashboard

12.5 Appointments

Symptom	Root cause	Fix
No slots shown at all	Business hours missing/null for the requested days; wrong timezone; existing appointments covering everything	Verify `business_hours` JSON; set the tenant timezone; check appointments table
Slots show but booking fails "no longer available"	Double-booking race or slot picked in the past	Regenerate slots; client refreshes the picker; the friendly fallback already handles it
Times are off by hours	Tenant `timezone` ≠ visitor's timezone; request passed `timezone=UTC` default	Set `timezone` per tenant (dashboard/settings); the widget passes no tz — align defaults
Buffer not applied	`buffer_minutes` set to 0 or not saved	Check tenant row; editor defaults 15

12.6 Email

Symptom	Root cause	Fix
No emails arrive (booking succeeds)	GAS URL wrong/not deployed; Gmail quota hit; OAuth not authorized	`python scripts/test_gas_email.py --to you@example.com` ; re-run `testEmail()` in Apps Script; check sent folder + quota
Emails land in spam	Gmail sender + HTML formatting	Ask recipients to mark not-spam; keep templates small; Gmail-to-Gmail usually fine

"Email send error" in logs	GAS endpoint 5xx/network, or Apps Script quota exception	Curl the /exec URL; read the error JSON; confirm the deployment is "Anyone" + "Execute as Me"
No customer confirmation email from the chat flow	The chat flow has no email capture, so it no longer sends a hardcoded placeholder email — it only notifies the business if `notification_email` is set	For customer confirmations from chat, capture the visitor's email (widget form / extraction prompt) and pass it to `send_appointment_confirmation`; the `/widget/appointments/{slug}` endpoint (which takes an email) already emails both parties

12.7 Auth / Admin

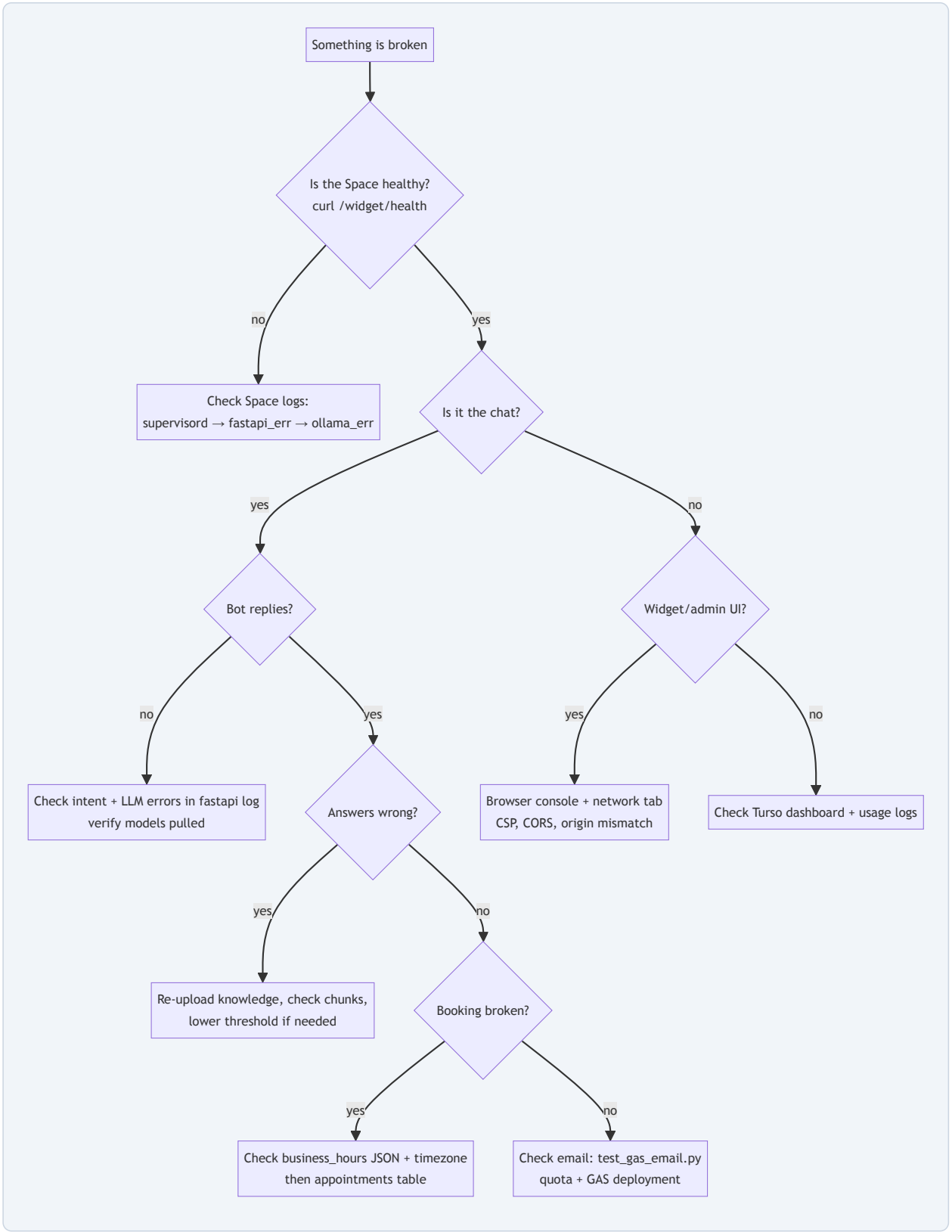
Symptom	Root cause	Fix
Admin login loops / "Couldn't sign you in"	Clerk app not configured for the pages.dev domain	Add the admin URL to Clerk → Domains (and allowed redirect URLs)
Signs in but admin API returns 401/403	Email ≠ `ADMIN_EMAIL` secret, or an old dashboard build without the token wrapper	Verify the `ADMIN_EMAIL` secret matches the signed-in account; current code attaches the token automatically via `api.ts` (9.1)
403 for valid user	`verify_aud: False` + audience mismatch	By design Clerk audience differs; keep the option off and rely on the email check
TenantDetail tabs empty (conversations/appointments)	Old revision — the page used tenant-scoped `/api/tenants/me/...` endpoints that needed an `X-Tenant-Slug` header the admin flow never sent	Current code uses admin-scoped endpoints (`/admin/tenants/{id}/conversations|appointments|knowledge`) with the token attached — rebuild and redeploy the admin dashboard
Brand color picked in "Add Tenant" never appears on the widget	Old revision — the front-end posted `color` but the API field is `primary_color`, and Pydantic silently dropped the unknown key	Current code sends `primary_color: newTenant.color` — rebuild and redeploy the admin dashboard
Dashboard Settings save does nothing	Old revision — `BusinessHoursEditor`'s `onSave` was a `// TODO: console.log`	Current code PATCHes `/admin/tenants/{id}` with hours/timezone/slot/buffer — rebuild and redeploy the admin dashboard

12.8 Deployment / CI

Symptom	Root cause	Fix
deploy-hf fails at "Log in to Hugging Face"	`HF_TOKEN` expired or lacks write scope	Regenerate token with Write access; update secret
Widget/admin deploy runs but pages.dev unchanged	Pages project name mismatch (`projectName:`) or branch mismatch	Match `projectName` to an existing Pages project; deploy branch `main`

First deploy takes 10+ min	Base image + pip + no layer cache	Normal on cold cache; subsequent builds reuse the registry buildcache
Keep-alive workflow shows red	Space sleeping (cold start) — expected	The `|| echo` swallows it; verify manually via `/widget/health` in a browser

12.9 Decision tree — where do I start?



12.10 Log analysis patterns

```
# The five strings that explain most incidents:
tail -f /var/log/supervisor/fastapi_err.log | grep -E "LLM error|Email send error|Traceback|ImportError"
tail -f /var/log/nginx/error.log | grep "limit_req"          # rate-limit rejections
tail -f /var/log/supervisor/ollama.log | grep -iE "error|out of memory"
# In Turso (dashboard → SQL):
SELECT event_type, COUNT(*) FROM usage_logs GROUP BY event_type;
SELECT status, COUNT(*) FROM appointments GROUP BY status;
SELECT source_id, COUNT(*) FROM knowledge_chunks GROUP BY source_id;
```

13. Scaling & Cost Optimization

This chapter is a map, not a plan: here's where the free tier ends, what breaks first, and the cheapest ways to buy headroom.

13.1 Free-tier limits — the hard ceilings

Service	Free limit	What you'll hit first
HF Spaces	16 GB RAM, 2 vCPU, sleeps after ~48 h idle	CPU latency as tenants multiply (all inference is serial per request)
Turso	9 GB storage, 1 B row reads/month	Reads: RAG searches + dashboard rollups dominate
Cloudflare Pages	Unlimited bandwidth (fair use), 100k requests/day on Workers functions	Practically never for static assets
Clerk	10k monthly active users	Never for a solo operator
Gmail via GAS	100 emails/day (personal)	Booking confirmations at ~100/day

13.2 Paid-upgrade triggers

Trigger	Upgrade	Cost
Chat response latency > 5 s average or > 3 concurrent chats	HF Space CPU upgrade, or move Ollama to a GPU instance	from ~$0.60/hr GPU
Turso reads > 80% of 1 B/mo	Turso scale plan	$29/mo (100 GB)
Gmail > 80 emails/day sustained	Google Workspace	$6/user/mo (1,500/day)
More than a handful of admins/agents	Clerk paid plan	$25/mo @ 10k+ MAU
You want HTTPS + custom domains per client	Cloudflare Pro (or keep free — Pages supports custom domains on free)	free, or $20/mo Pro for advanced rules

13.3 Architecture changes for scale

1. **More uvicorn workers.** `--workers 1` is fine until concurrent chats queue on CPU. Bump to `--workers 2` (matches 2 vCPU) and ensure libSQL's async client tolerates multiple connections (it does). Ollama stays single-server — it serializes inference anyway.
2. **Larger/faster model.** When answer quality matters more than cost, swap `phi3:mini` → `phi3:medium` / `llama3.1:8b` via `OLLAMA_CHAT_MODEL` … but mind the 4K context and RAM. The `max_tokens` /temperature table in Chapter 5 stays valid; only latency grows.
3. **Embedding model change** requires rebuilding `knowledge_vec` (dimension is baked into the virtual table) — re-upload or re-embed all chunks.

4. **Cache hot paths.** `/widget/config/{slug}` is immutable-ish and fetched on every page load — serve it with nginx `proxy_cache` or a TTL header. Chat history is read often — add the missing `/widget/history` endpoint (see below) with `LIMIT` + cursor pagination.
5. **Transactional booking.** Wrap the conflict check + insert in a transaction (or add `UNIQUE(tenant_id, start_time)` and treat constraint violations as "taken") to close the double-booking race at scale.
6. **Email queue.** Replace the fire-and-forget POST with a small `outbox` table + retry worker (3 attempts, exponential backoff) — this is the single highest-value reliability upgrade for a booking business.
7. **Intent/history gaps.** The codebase has clear TODOs: the `/widget/history` route the widget already calls (`loadHistory`), the real email capture in chat bookings, and wiring the business-hours editor to the API. Land these before chasing performance.

13.4 Multi-region considerations

- **Turso locations:** create the database in the region closest to your clients (`turso db create --location`); reads/writes are single-region today — a latency of ~100–200 ms per query is fine for chat, noticeable for dashboards.
- **HF Spaces region** is fixed per Space; there's no multi-region for the container. The pragmatic multi-region story: keep the API in one region and let Cloudflare's edge cache the static widget; chat latency for distant clients is dominated by the single Ollama hop anyway.
- **If you outgrow Spaces:** the Docker image is portable — same image on Fly.io/Railway/Cloud Run with a real domain and autoscaling; nginx/supervisord/healthcheck configs port as-is.

13.5 Cost-optimization checklist (keep $0)

- ✅ Keep the Space awake with the keep-alive cron (avoids 30 s cold starts costing you nothing but user patience).
- ✅ Reuse the Docker buildcache (already configured) — rebuilds cost CI minutes, not money.
- ✅ Watch Turso reads monthly; add caching before paying for reads.
- ✅ Batch embeddings during onboarding (already one `embed(texts)` call) — never embed one chunk at a time.
- ✅ Deploy only on relevant paths (already in workflows) — a docs edit shouldn't trigger a 10-min image build.
- ✅ Use Cloudflare Pages custom domains (free) to give each client `chat.yourclient.com` — a $0 feature that looks expensive.

> **BOTTOM LINE.** The architecture's genius is that every scaling lever is a *config change or a small feature*, not a rewrite: workers, model name, threshold, cache headers, a unique constraint, an outbox table. The free tier buys you thousands of clients of runway; the paid triggers are clear and cheap when you cross them.

Appendix — Migration Roadmap: Phases 1–4 in Detail

Why this appendix exists: the original system deployed its backend as a Docker Space on Hugging Face. In July 2026 Hugging Face moved Docker (and Gradio) Spaces behind a paid PRO subscription, leaving only Static Spaces free. The project was re-planned around a **$0 Cloudflare stack** and is being ported in phases (tracked in `hf-docker-exit-spec.md`). The chapters above describe the reference implementation; this appendix is the source of truth for what is live now and what ships next.

Phase	What ships	Where (code)	Done when
1 — Chat + RAG ✅ *implemented*	Widget chat (SSE), RAG answers from per-tenant knowledge, conversation history, admin-lite tenant + knowledge ingest, GitHub Actions deploys	`worker/` — `config.ts`, `db.ts`, `llm.ts`, `rag.ts`, `chat.ts`, `clerk.ts`, `admin.ts`, `index.ts`; schema in `worker/db/schema.sql`	Widget on a test page answers from knowledge; `/health` green
2 — Appointments + email	Availability slots, booking/cancel in the chat, business-hours parsing, Google Apps Script email confirmations, `email_pending` retry UX on Gmail quota	Worker ports of `backend/app/services/appointments.py` + `email.py` (Chapter 6/7 detail); `gas-email/`	"Book Tuesday 2pm" books + emails both sides
3 — Full admin dashboard	Tenant CRUD, analytics, conversations, appointments, usage, PDF upload (browser-side `pdfjs-dist` extraction — the Worker can't spend CPU on PDFs), Clerk-only auth (bootstrap token removed)	Admin APIs in `worker/src/admin.ts`; `admin-dashboard/` wired via `VITE_API_URL` (already supported in `api.ts`)	Admin UI matches the feature list in Chapter 9
4 — Cleanup & docs	Remaining docs/guides rewritten, legacy `scripts/` (Chapter 10) ported to Worker API calls, old FastAPI `backend/` removed once parity is proven	Repo root	Repo has no Hugging Face / Docker remnants

Key decisions locked in the spec (hf-docker-exit-spec.md)

- **LLM:** Gemini free tier — flash-class chat model via `CHAT_MODEL` (verify free RPD in AI Studio; limits are volatile and treated as config), embeddings via `gemini-embedding-001` @ 768 dims so the vec0 `FLOAT[768]` table and float32 blobs need no migration. Groq / Workers-AI are optional failover providers (`LLM_CHAT_PROVIDERS`). §5.3–5.5.
- **Provider-mixing rule:** never embed new chunks with a different model while existing vectors in the same table come from another — cross-model cosine distances are meaningless. Embeddings "failover" = full re-embed job or graceful degradation, never a silent switch. §5.3, §5.6.
- **Database:** Turso stays; sqlite-vec is built in server-side, so the Chapter 3 SQL works from the Worker via `@libsql/client`. Schema: `worker/db/schema.sql`.

- **PDFs (§5.6):** the Worker free plan caps CPU at 10 ms/request, so PDF text extraction happens outside it — in the admin browser (`pdfjs-dist`) or a local CLI — then POSTs text to `/knowledge/text` . `unpdf` is a possible future in-Worker path for small files.
- **Auth (§5.4):** Clerk JWKS verification in the Worker (WebCrypto); optional `ADMIN_BOOTSTRAP_TOKEN` only for initial onboarding, deleted in Phase 3.
- **Graceful limits (§5.7):** self-throttle + daily caps; 429 → retry → failover → friendly "at my usage limit" message; embeddings quota surfaces as HTTP 429 `embed_quota` in the admin.

Mapping this guide's chapters to the new code

Chapters 3–13 keep their value as the design + reference documentation for `backend/` (still in the repo). Quick map: Ch. 4 (FastAPI) → `worker/src/index.ts` + handlers; Ch. 5 (AI pipeline) → `worker/src/llm.ts` / `rag.ts` (Gemini instead of Ollama); Ch. 8 (widget) → unchanged plus the `data-api-url` attribute; Ch. 9 (admin/Clerk) → API arrives in Phase 3; Ch. 11–13 (monitoring/scaling) → replace HF-Space concerns with Workers/Gemini free-tier budget (§5.5). The live build steps always live in `DEPLOY.md` .
